

ЗАДАЧИ ЗА САМОПОДГОТОВКА
ПО
Увод в програмирането,
Обектно-ориентирано програмиране,
Структури от данни и Функционално
програмиране

Калин Георгиев
kalin@fmi.uni-sofia.bg

13 май 2026 г.

Съдържание

1	Увод в програмирането	7
1	Увод, основи и примери	7
1.1	Основни примери	7
1.2	Променливи, вход и изход, логически и аритметични операции, условен оператор	7
1.3	Цикли	8
1.4	Машины с неограничени регистри	9
2	Типове и функции	12
2.1	Прости примери за функции	12
2.2	Елементарна растерна графика	12
3	Цикли, масиви и низове	18
3.1	Цикли II	18
3.2	Цикли и низове	18
3.3	Матрици и вложени цикли	20
3.4	Елементарно сортиране на масиви	20
3.5	Низове II	22
3.6	Инвариант на цикъл и верификация на програми . .	23
4	Указатели и програмен стек	31
4.1	Предаване на масиви и указатели като параметри на функции	31
4.2	Масиви от указатели	31
4.3	Програмен стек	31
5	Рекурсия	33
5.1	Прости рекурсивни функции	33
5.2	Търсене с пълно изчерпване	34
6	Структури	39
7	Динамична памет	42
7.1	Заделяне на динамична памет	42
7.2	Масиви от структури в динамичната памет и текстови файлове	43

7.3	Динамични масиви	43
8	Шаблони и указатели към функции	46
8.1	Шаблони на функции	46
8.2	Функции от високо ниво	47
2	Обектно ориентирано програмиране	49
9	Класове I: Прости класове, методи и оператори	49
10	Жизнен цикъл на обектите	51
10.1	Конструктори	51
10.2	Управление на динамична памет	53
11	λ функции	56
11.1	Map и Reduce	56
11.2	λ функции и <code>std::function</code>	57
12	Линейни едносвързани списъци	60
13	Клас символен низ	62
14	Наследяване и полиморфизъм	64
14.1	Наследяване, виртуални методи и абстрактни класове	64
14.2	Сериализация и десериализация на контейнери. Шаблон “Фабрика”	68
14.3	Виртуални конструктори и деструктори	70
14.4	Наследяване и агрегация. Йерархични дървета от обекти, съдържащи други обекти. Шаблон за дизайн “Посетител.”	71
15	Рекурсивен парсер	74
16	Обработка на изключения	76
3	Структури от данни	77
16	Линеен едносвързан и двусвързан списък	77
16.1	Представяне на двусвързан списък	77
16.2	Списъци и сложности	79
16.3	SList (опростен вариант на Skip List)	79
17	Итератори, интервали и адаптери	81
17.1	Итератори за линейни СД	81
17.2	Интервали и адаптери	82
18	Приложения на структурата от данни стек	86
18.1	Общи задачи за стекове	86
18.2	Изрази и стекове	86
18.3	Цикъл със стек вместо рекурсия	86
19	Двоични дървета	89
19.1	Прости обхождания	89
19.2	Извеждане и сериализация	90

19.3	Обхождания	92
19.4	Представяне на аритметичен израз чрез дърво	94
19.5	Построяване и модификации на дърво	95
20	Префиксно дърво (Trie)	98
21	Изображения (Maps)	100
21.1	HashMap	100
4	Функционално програмиране	103
22	Уводни задачи	103
23	Проста рекурсия над списъци	104
24	Конструиране на списъци	105
25	Работа със списъци и образци	106
26	Дефиниране на алгебрични типове	107
27	Монадът Maybe	108
28	Функции от по-висок ред	109
29	λ функции	111
30	Вход/Изход	113
31	Индуктивни типове данни	115
32	Оценка на (аритметичен) израз	116
33	Двоични дървета	118
33.1	Прости обхождания	118
33.2	Извеждане и сериализация	119
33.3	Обхождания	119
33.4	Decision Tree	120
33.5	Други	123
34	Наследяване	123

Някои от задачите по-долу са решени в сборника [1] *Магдалина Тодорова, Петър Армянов, Дафина Петкова, Калин Георгиев, "Сборник от задачи по програмиране на C++. Първа част. Увод в програмирането"*. За задачите от сборника е посочена номерацията им от сборника.

Глава 1

Увод в програмирането

1 Увод, основи и примери

1.1 Основни примери

- 1.1. Превърнете рождената си дата шестнадесетична, в осмична и в двоична бройни системи.
- 1.2. Как бихте кодирали вашето име само с числа? Измислете собствено представяне на символни константи чрез редици от числа и запишете името си в това представяне.

Разгледайте стандартната ASCII таблица (<http://www.asciitable.com/>) и запишете името си чрез серия от ASCII кодове.

1.2 Променливи, вход и изход, логически и аритметични операции, условен оператор

- 1.3. Задача 1.6.[1] Да се напише програма, която по зададени навършени години намира приблизително броя на дните, часовете, минутите и секундите, които е живял човек до навършване на зададените години.
- 1.4. Задача 1.7.[1] Да се напише програма, която намира лицето на триъгълник по дадени: а) дължини на страна и височина към нея; б) три страни.
- 1.5. Задача 2.7.[1] Да се напише програма, която въвежда координатите на точка от равнина и извежда на кой квадрант принадлежи тя. Да се разгледат случаите, когато точката принадлежи на някоя от координатните оси или съвпада с центъра на координатната система.
- 1.6. Задача 1.14.[1] Да се запише булев израз, който да има стойност истина, ако посоченото условие е вярно и стойност - лъжа, в противен случай:

- а) цялото число p се дели на 4 или на 7;
- б) уравнението $ax^2 + bx + c = 0 (a \neq 0)$ няма реални корени;
- в) точка с координати (a, b) лежи във вътрешността на кръг с радиус 5 и център $(0, 1)$; г) точка с координати (a, b) лежи извън кръга с център (c, d) и радиус f ;
- г) точка принадлежи на частта от кръга с център $(0, 0)$ и радиус 5 в трети квадрант;
- д) точка принадлежи на венеца с център $(0, 0)$ и радиуси 5 и 10;
- е) x принадлежи на отсечката $[0, 1]$;
- ж) x е равно на $\max \{a, b, c\}$;
- з) x е различно от $\max \{a, b, c\}$;
- и) поне една от булевите променливи x и y има стойност `true`;
- к) и двете булеви променливи x и y имат стойност `true`;
- л) нито едно от числата a, b и c не е положително;
- м) цифрата 7 влиза в запис на положителното трицифрено число p ;
- н) цифрите на трицифреното число m са различни;
- о) поне две от цифрите на трицифреното число m са равни помежду си;
- п) цифрите на трицифреното естествено число x образуват строго растяща или строго намаляваща редица;
- р) десетичните записи на трицифрените естествени числа x и y са симетрични;
- с) естественото число x , за което се знае, че е по-малко от 23, е просто.

1.7. Задача 2.12.[1] Да се напише програма, която проверява дали дадена година е високосна.

1.3 Цикли

- 1.8. Задача 1.20.[1] Да се напише програма, която по въведени от клавиатурата цели числа x и k ($k \geq 1$) намира и извежда на екрана k -тата цифра на x . Броенето да е отдясно наляво.
- 1.9. Задача 2.40.[1] Да се напише програма, която (чрез цикъл `for`) намира сумата на всяко трето цяло число, започвайки от 2 и ненадминавайки n (т.е. сумата $2 + 5 + 8 + 11 + \dots$).
- 1.10. Задача 2.44.[1] Дадено е естествено число n ($n \geq 1$). Да се напише програма, която намира броя на тези елементи от серията числа $i^3 + 13 \times i \times n + n^3$, $i = 1, 2, \dots, n$, които са кратни на 5 или на 9.

- 1.11. За въведени от клавиатурата естествени числа n и k , да се провери и изпише на екрана дали n е точна степен на числото k .

Упътване: Разделете променливата n на променливата k “колкото пъти е възможно” и проверете дали n достига единица или някое друго число след края на процеса. Използвайте добре подбрано условие за `for` цикъл, оператора `%` за намиране на остатък при целочислено деление, и оператора за целочислено деление `/`.

1.4 Машини с неограничени регистри

Дефиницията на Машина с неограничени регистри по-долу е взaimствана от учебника [2] А. Дичев, И. Сосков, “Теория на програмите”, Издателство на СУ, София, 1998.

“Машина с неограничени регистри” (или МНР) наричаме абстрактна машина, разполагаща с неограничена памет. Паметта на машината се представя с безкрайна редица от естествени числа $m[0], m[1], \dots$, където $m[i] \in \mathcal{N}$. Елементите $m[i]$ на редицата наричаме “клетки” на паметта на машината, а числото i наричаме “адрес” на клетката $m[i]$.

МНР разполага с набор от инструкции за работа с паметта. Всяка инструкция получава един или повече параметри (операнди) и може да предизвика промяна в стойността на някоя от клетките на паметта. Инструкциите на МНР за работа с паметта са:

- 1) **ZERO n** : Записва стойността 0 в клетката с адрес n
- 2) **INC n** : Увеличава с единица стойността, записана в клетката с адрес n
- 3) **MOVE x y** : Присвоява на клетката с адрес y стойността на клетката с адрес x

“Програма” за МНР наричаме всяка последователност от инструкции на МНР и съответните им операнди. Всяка инструкция от програмата индексирате с поредния ѝ номер. Изпълнението на програмата започва от първата инструкция и преминава през всички инструкции последователно, освен в някои случаи, описани по-долу. Изпълнението на програмата се прекратява след изпълнението на последната ѝ инструкция. Например, след изпълнението на следната програма:

```
0: ZERO 0
1: ZERO 1
```

```

2: ZERO 2
3: INC 1
4: INC 2
5: INC 2

```

Първите три клетки на машината ще имат стойност 0, 1, 2, независимо от началните им стойности.

Освен инструкциите за работа с паметта, МНР притежават и една инструкция за промяна на последователността на изпълнение на програмата:

- 4) JUMP x : Изпълнението на програмата “прескача” и продължава от инструкцията с пореден номер x . Ако програмата има по-малко от $x + 1$ инструкции, изпълнението ѝ се прекратява
- 5) JUMP x y z : Ако съдържанията на клетките x и y съвпадат, изпълнението на програмата “прескача” и продължава от инструкцията с пореден номер z . В противен случай, програмата продължава със следващата инструкция. Ако програмата има по-малко от $z + 1$ инструкции, изпълнението ѝ се прекратява

Например, нека изпълнението на следната програма започва при стойности на клетките на паметта 10,0,0,...:

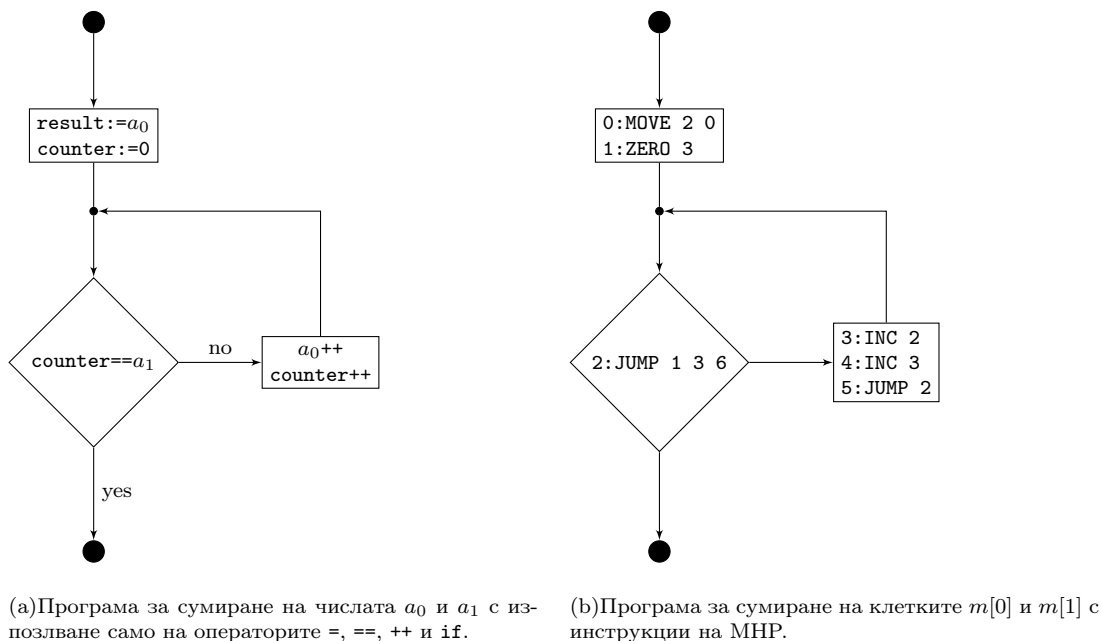
```

0: JUMP 0 1 5
1: INC 1
2: INC 2
3: INC 2
4: JUMP 0

```

След приключване на програмата, първите три клетки на машината ще имат стойности 10, 10, 20.

- 1.12. Нека паметта на МНР е инициализирана с редицата $m, n, 0, 0, \dots$. Да се напише програма на МНР, след изпълнението на която клетката с адрес 2 съдържа числото $m + n$.
- 1.13. Нека паметта на МНР е инициализирана с редицата $m, n, 0, 0, \dots$. Да се напише програма на МНР, след изпълнението на която клетката с адрес 2 съдържа числото $m \times n$.
- 1.14. Нека паметта на МНР е инициализирана с редицата $m, n, 0, 0, \dots$. Да се напише програма на МНР, след изпълнението на която клетката с адрес 2 съдържа числото 1 тогава и само тогава, когато $m > n$ и числото 0 във всички останали случаи.



Фигура 1.1: Блок схеми на програма за сумиране на числа

Упътване: На Фигура 1.1 (а) е показана блок схема на програма, използваща само операторите $=$, $==$, $++$ и if , която намира в променливата **result** сумата на променливите a_0 и a_1 . a_0 и a_1 считаме за дадени. Променливата **count** се инициализира с 0, а **result** - с a_0 . В цикъл се добавя по една единица към **count** и **result** дотогава, докато **count** достигне стойността на a_1 . По този начин, към **result** се добавят a_1 на брой единици, т.е. стойността ѝ се увеличава с a_1 спрямо началната ѝ стойност a_0 .

На Фигура 1.1 (b) е показана същата програма, като операторите от първата са заменени със съответните им инструкции на МНР. Резултатът от програмата се получава в клетката $m[2]$, а за брояч се ползва клетката $m[3]$. На блок схемата са дадени поредните номера на инструкциите в окончателната програмата на МНР:

```
0:MOVE 2 0
1:ZERO 3
2:JUMP 1 3 6
3:INC 2
4:INC 3
5:JUMP 2
```

2 Типове и функции

2.1 Прости примери за функции

- 2.1. Задача 4.12.[1] Да се напише булева функция, която проверява дали дата, зададена в следния формат: dd.mm.yyyy е коректна дата от грегорианския календар.
- 2.2. Задача 4.25.[1] Да се дефинира процедура, която получава целочислен параметър n и база на бройна система $k \leq 16$. Процедурата да отпечата на екрана представянето на числото n в системата с база k .
- 2.3. Задача 2.57.[1] Да се напише булева функция, която проверява дали сумата от цифрите на дадено като параметър положително цяло число е кратна на 3.
- 2.4. Задача 2.81.[1] Едно положително цяло число е свършено, ако е равно на сумата от своите делители (без самото число). Например, 6 е свършено, защото $6 = 1 + 2 + 3$; числото 1 не е свършено. Да се напише процедура, която намира и отпечата на екрана всички свършени числа, ненадминаващи дадено положително цяло число в параметър n .

2.2 Елементарна растерна графика

Следните задачи да се решат с показаните на лекции графични примитиви, базирани на платформата за компютърни игри SDL2. За целта е необходимо да инсталирате SDL2 на компютъра си и да настроите средата си за програмиране така, че да свърже SLD2 с вашия проект. Информация за това можете да намерите на сайта на платформата. Задачите можете да решите с помощта на всяка друга библиотека, поддържаща примитивите за рисуване на точки и отсечки.

Примерната програма от лекции използва модула **Sdlwrapper**, който можете да намерите в хранилището на курса:

```
#include "draw/sdlwrapper.h"
```

Sdlwrapper “обвива” библиотеката SLD2 и дефинира следните лесни за използване примитиви:

- **setColor (r,g,b)**: Дефинира цвят на рисуване с компоненти $r, g, b \in [0, 255]$. Например, белият цвят се задава с (255, 255, 255), червеният с (255, 0, 0) и т.н.
- **drawPixel(x,y)**: Поставя една точка на екранни координати (x, y) .

- `drawLine (x1,y1,x2,y2)`: Рисува отсечка, свързваща точките с екранни координати (x_1, y_1) и (x_2, y_2) .
- `updateGraphics()`: Извиква се веднъж в края на програмата, за да се изобрази нарисуваното с горните примитиви.

2.5. Да се нарисуват програмно следните фигури чрез дефиниране на подходящи функции:

- Равностранен триъгълник по дадена дължина на страната
- Равностранен шестоъгълник по дадена дължина на страната
- Графиката на функциите $y = \sin(x)$, $y = x^2$, и $y = x^3 + 5x^2$ поотделно и заедно
- Равностранен многоъгълник по дадени координати на пресечната точка на симетралите му (център), брой страни n и разстояние от центъра до върховете r . При какви стойности на n фигурата наподобява окръжност?
- Логаритмична крива
- Елипса с център дадени (x, y) и радиуси дадени r_1 и r_2

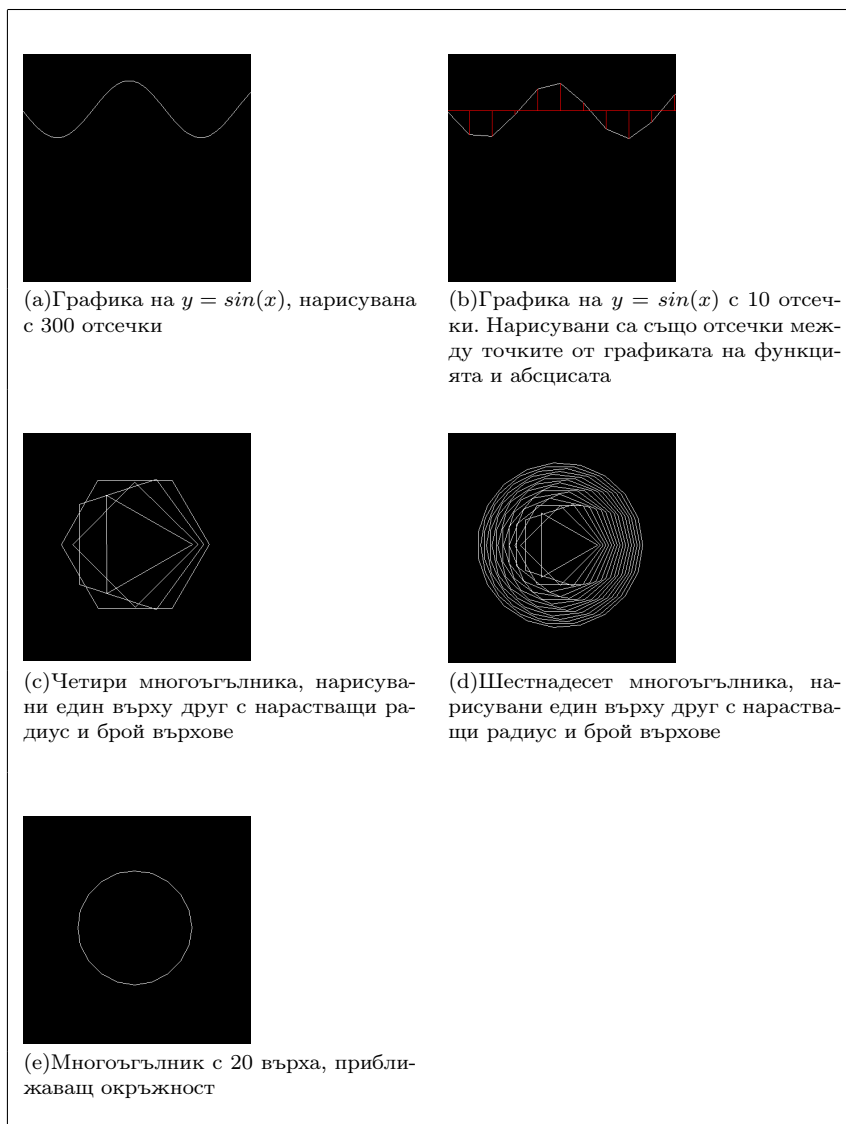
2.6. По дадени екранни координати (x, y) на горния ляв ъгъл на квадрат, дължина на страната a на квадрата и число n :

- Да се нарисува квадратна матрица от $n \times n$ квадрата със страна a/n , изпълваща дадения квадрат.
- Квадратите от горното условие да се заменят с триъгълниците, образувани от пресичането на диагоналите им.

Упътване към задачата за чертане на графика на функция.

Да разгледаме функцията $y = \sin(x)$: Рисуват се отсечки между последователни точки от графиката на функцията, като всяка следваща точка се получава като увеличаваме стойността на аргумента x с числото `stepX`. Добре е `stepX` да е достатъчно малко, за да не се “накъса” графиката на функцията и да е достатъчно голямо, за да е видимо изменението ѝ. За този пример сме избрали `stepX = 0.05`.

Тъй като $\sin(x) \in [-1, 1]$, ако директно визуализираме точките на получените по този начин координати $(x, \sin(x))$, те ще са “сгъстени” около правата $y = 0$ и резултатът няма да е добър. Поради това, координатите на получените точки от кривата се умножават по `scaleX` и `scaleY` съответно, $(x * \text{scaleX}, \sin(x) * \text{scaleY})$, за да се “разпъне” графиката по двете оси. (Вж. Фигура 1.2)



Фигура 1.2: Примерни резултати от решенията на някои задачи

```

const double //scaleX: коефициент на скалиране по X
scaleX = 40.0,
//y0: ордината на началната точка
y0 = 100,
//scaleY: коефициент на скалиране по Y
scaleY = 50.0,
//stepX: стъпка за нарастване на аргумента
stepX = 0.05;
//nsegments: брой сегменти от кривата
const int nsegments = 300;

for (int i = 0; i < nsegments; i++)
{
    double x      = scaleX*i*stepX,
           xnext  = scaleX*(i+1)*stepX,
           y      = y0+scaleY*sin(stepX*i),
           ynext  = y0+scaleY*sin(stepX*(i+1));
    drawLine (x,y,xnext,ynext);
}

```

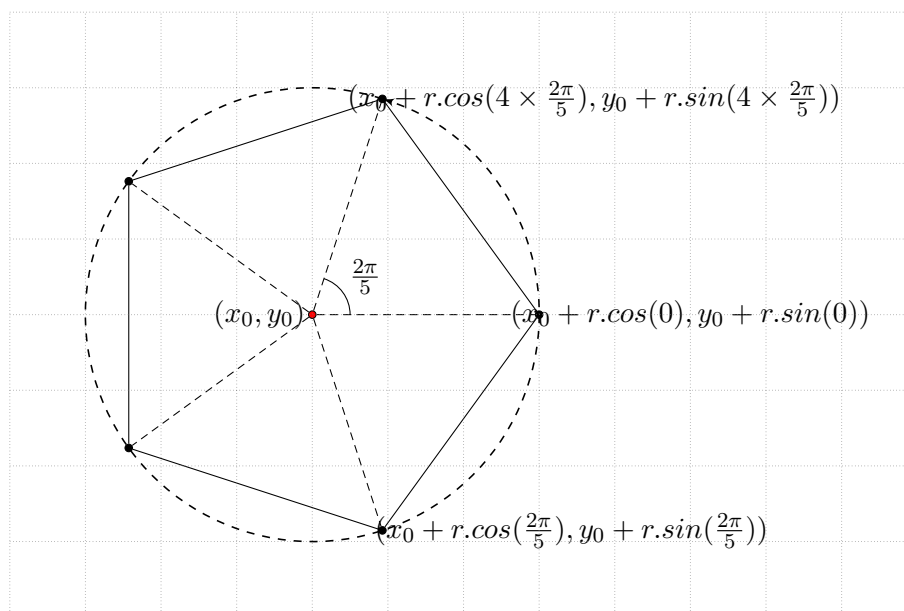
Упътване към задачата за чертане на многоъгълник: Върховете на многоъгълника получаваме, като започнем с точката с координати $(x + radius, y)$ и получаваме всяка следваща точка като “завъртим” предишната около (x, y) с $2\pi/n$ радиана, където n е броят на върховете на многоъгълника. Получените по този начин точки се съединяват с отсечки. (Вж. Фигура 1.2 и Фигура 1.3.)

```

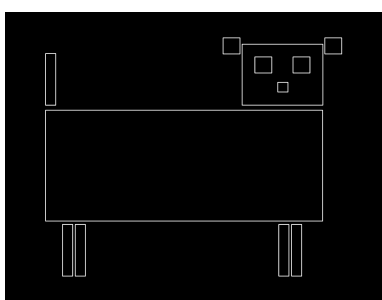
/*
Функция polygon (n,x,y,radius): Рисуване на многоъгълник
параметър n: брой върхове на многоъгълника
параметри x,y: координати на центъра на многоъгълника
параметър radius: разстояние от центъра до върховете
*/
void polygon (int n, double x, double y, double radius)
{
    for (int side = 0; side < n; side++)
    {
        drawLine (radius*cos(side*2.0*M_PI/n)+x,
                  radius*sin(side*2.0*M_PI/n)+y,
                  radius*cos((side+1)*2.0*M_PI/n)+x,
                  radius*sin((side+1)*2.0*M_PI/n)+y);
    }
}

```

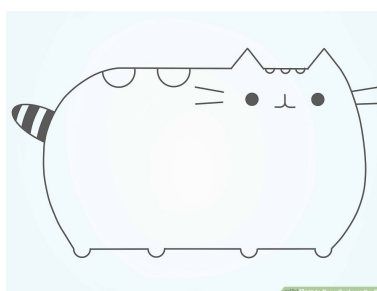
- 2.7. Да се нарисова програмно фигура на котка. (вж. примерите на фигура 1.4 и статията [3]).
- 2.8. (*) Следната задача илюстрира метода на трапеците (*Trapezoidal rule*) за приближено изчисление на определени интеграли: Да се нарисуват



Фигура 1.3: Рисуване на петогълник чрез намиране на 5 равноотдалечени точки по окръжността с радиус r и център (x_0, y_0)



Фигура 1.4: Котка, нарисувана само с правоъгълници



Фигура 1.5: Pusheen the cat. Фигурата е от [3]

програмно координатни оси на евклидова координатна система с даден център в екранните координати (x,y) . Да приемем, че в програмата е дефинирана функция `double f (double x)`, за която знаем, че е дефинирана за всяка стойност на x .

- Да се изобрази графиката на функцията спрямо нарисувана координатна система
- Да се приближи чрез трапеци с дадена дължина на основата δ фигурата, заключена между видимата графика на фигурата и абсцисата
- Да се визуализират така получените трапеци
- Да се изчисли сумата от лицата на така получените трапеци
- Да се експериментира с различни дефиниции на функцията f

За пример виж фигура 11.2.

3 Цикли, масиви и низове

3.1 Цикли II

Където не е посочено изрично, под “редица от числа $a_0, a_1, \dots, a_{n-1}$ ” по-долу се разбира последователност от n числа, въведени от стандартния вход. Задачите да се решат *без* използването на масиви.

- 3.1. Задача 3.1. [1] Да се напише програма, която въвежда редица от n цели числа ($1 \leq n \leq 50$) и намира и извежда минималното от тях.
- 3.2. Задача 3.2. [1] Да се напише програма, която въвежда редицата от n ($1 \leq n \leq 50$) цели числа $a_0, a_1, \dots, a_{n-1}$ и намира и извежда сумата на тези елементи на редицата, които се явяват удвоени нечетни числа.
- 3.3. Задача 3.3. [1] Да се напише програма, която намира и извежда сумата от положителните и произведението на отрицателните елементи на редицата от реални числа $a_0, a_1, \dots, a_{n-1}$ ($1 \leq n \leq 20$).
- 3.4. Задача 3.7. [1] Да се напише програма, която изяснява има ли в редицата от цели числа $a_0, a_1, \dots, a_{n-1}$ ($1 \leq n \leq 100$) поне два последователни елемента с равни стойности.
- 3.5. Задача 3.8. [1] Да се напише програма, която проверява дали редицата от реални числа $a_0, a_1, \dots, a_{n-1}$ ($1 \leq n \leq 100$) е монотонно растяща.
- 3.6. Задача 3.15. [1] Да се напише програма, която въвежда реалните вектори $a_0, a_1, \dots, a_{n-1}$ и $b_0, b_1, \dots, b_{n-1}$ ($1 \leq n \leq 100$), намира скаларното им произведение и го извежда на екрана.
- 3.7. Задача 3.10. [1] Да се напише програма, която за дадена числова редица $a_0, a_1, \dots, a_{n-1}$ ($1 \leq n \leq 100$) намира дължината на най-дългата ѝ ненамаляваща подредица $a_i, a_{i+1}, \dots, a_{i+k}$ ($a_i \leq a_{i+1} \leq \dots \leq a_{i+k}$).

3.2 Цикли и низове

Където не е посочено изрично, под “редица от символи $s_0, s_1, \dots, s_{n-1}$ ($1 \leq n \leq 100$) $a_0, a_1, \dots, a_{n-1}$ ” по-долу се разбира символен низ с дължина n , въведен от клавиатурата в масив от тип `char[255]`.

- 3.8. Задача 3.11. [1] Дадена е редицата от символи $s_0, s_1, \dots, s_{n-1}$ ($1 \leq n \leq 100$). Да се напише програма, която извежда отначало всички символи, които са цифри, след това всички символи, които са малки латински букви и накрая всички останали символи от редицата, запазвайки реда им в редицата.

- 3.9. Задача 3.13. [1] Задача 3.13. Да се напише програма, която определя дали редицата от символи $s_0, s_1, \dots, s_{n-1}$ ($1 \leq n \leq 100$) е симетрична, т.е. четена отляво надясно и отдясно наляво е една и съща.
- 3.10. Да се напише функция, която по два низа намира дължината на най-дългия им общ префикс. *Префикс на низ наричаме подниз със същото начало като дадения. Пример: празният низ и низовете "a", "ab", и "abc" са всички възможни префикси на низа "abc". Дължината на най-дългия общ префикс на низовете "abcde" и "abcuwz" е 3.*
- 3.11. Да се напише функция, която в даден низ замества всички малки латински букви със съответните им големи латински букви.
- 3.12. Да се напише функция `reverse(s)`, която превръща даден низ в огледалния му образ. *Например, низът "abc" ще се преобразува до "cba".*
- 3.13. Да се напише функция, която по даден низ s , всички букви в който са латински, извършва следната манипулация над него: Ако s съдържа повече малки, отколкото големи букви, замества всички големи букви в s с малки. В останалите случаи, всички малки букви се заместват с големи.
- 3.14. Задача 3.26. "Хистограма на символите"[1] Символен низ е съставен единствено от малки латински букви. Да се напише програма, която намира и извежда на екрана броя на срещанията на всяка от буквите на низа.
- 3.15. Да се напише булева функция, която по дадени низове s_1 и s_2 проверява дали s_2 е подниз на s_1 (*Например, низът "uv" е подниз на низовете "abwvc", "uvw", "ziv" и "uv", но не е подниз на низа "uwv".*).
- 3.16. Задача 3.28. "Търсене на функция"[1] Дадени са два символни низа с еднаква дължина s_1 и s_2 , съставени от малки латински букви. Да се напише програма, която проверява дали съществува функция $f : \text{char} \rightarrow \text{char}$, изобразяваща s_1 в s_2 , така че $f(s_1[i]) = s_2[i]$ и $i = 1..дължината на s_1 и s_2 . Упътване: За да е възможна такава функция, не трябва в s_1 да има символ, на който съответстват два или повече различни символи в s_2 . Например, низът "aba" може да бъде изобразен в низа "zwz", но не и в низа "zwi".$

3.3 Матрици и вложени цикли

- 3.17. Задача 3.18. [1] Дадени са числовите редици $a_0, a_1, \dots, a_{n-1}$ и $b_0, b_1, \dots, b_{n-1}$ ($1 \leq n \leq 50$). Да се напише програма, която въвежда от клавиатурата двете редици и намира броя на равенствата от вида $a_i = b_j$ ($i = 0, \dots, n-1, j = 0, \dots, n-1$).
- 3.18. Задача 3.21. [1] Две числови редици си приличат, ако съвпадат множествата от числата, които ги съставят. Да се напише програма, която въвежда числовите редици $a_0, a_1, \dots, a_{n-1}$ и $b_0, b_1, \dots, b_{n-1}$ ($1 \leq n \leq 50$) и установява дали си приличат.
- 3.19. Задача 3.29. [1] Дадена е квадратна целочислена матрица A от n -ти ред ($1 \leq n \leq 50$). Да се напише програма, която намира сумата от нечетните числа под главния диагонал на A (без него).
- 3.20. Задача 3.45. [1] Матрицата A има седлова точка в $a_{i,j}$, ако $a_{i,j}$ е минимален елемент в i -тия ред и максимален елемент в j -тия стълб на A . Да се напише програма, която извежда всички седлови точки на дадена матрица A с размерност $n \times m$ ($1 \leq n \leq 20, 1 \leq m \leq 30$).
- 3.21. Задача 3.113. (периодичност на масив). [1] Да се напише програма, която проверява дали в едномерен масив от цели числа съществува период. Например, ако масивът е с елементи 1, 2, 3, 1, 2, 3, 1, 2, 3, 1, периодът е 3. Ако период съществува, да се изведе.
- 3.22. Да се напише програма, която въвежда от клавиатурата матриците от цели числа $A_{N \times M}$ и $B_{M \times N}$ и извежда на екрана резултатът от умножението на двете матрици.

3.4 Елементарно сортиране на масиви

- 3.23. Да се реализира функция, сортираща масив по метода на мехурчето. При този метод масивът $a[0], \dots, a[n-1]$ се обхожда от началото към края, като на всяка стъпка се сравняват двойката съседни елементи $a[i]$ и $a[i+1]$. Ако $a[i+1] < a[i]$, местата им се разменят. Този процес се извършва n пъти.
- 3.24. Да се напише функция **unsortedness**, която оценява доколко един масив е несортиран като преброява колко от елементите му “не са на местата си”. Т.е. функцията намира броя на тези елементи a_i , които не са i -ти по големина в масива. Например, за масива $\{0, 2, 1\}$ това число е 2.
- 3.25. Да се напише функция **swappable**, която за масива $a[0], \dots, a[n-1]$ проверява дали има такова число i ($0 < i < n-1$), че масива $a_i, \dots, a_n, a_0, \dots, a_{i-1}$ е сортиран в нарастващ ред. Т.е. може ли масивът да се раздели на две

части (незадължително с равна дължина) така, че ако частите се разменят, да се получи нареден масив. Пример за такъв масив е {3, 4, 5, 1, 2}.

Функцията `std::clock()` от `<ctime>` връща в абстрактни единици времето, което е изминало от началото на изпълнение на програмата. Обикновено тази единица за време, наречена “tick”, е фиксиран интервал “реално” време, който зависи от хардуера на системата и конфигурацията ѝ. Константата `CLOCKS_PER_SEC` дава броя tick-ове, които се съдържат в една секунда реално време.

Чрез следния примерен код може да се измери в милисекунди времето за изпълнение на програмния блок, обозначен с “...”.

```
clock_t start = std::clock();  
//...  
clock_t end = std::clock();  
  
long milliseconds = (double)(end-start)/  
                    (CLOCKS_PER_SEC/1000.0);
```

Функцията `rand()` от `<cstdlib>` генерира редица от псевдо-случайни числа. Всяко последователно изпълнение на функцията генерира следващото число от редицата. За да се осигури, че при всяко изпълнение на програмата ще се генерира различна редица от псевдо-случайни числа, е необходимо да се изпълни функцията `srand()` с параметър, който е различен за всяко изпълнение на програмата. Една лесна възможност е да се ползва резултата на функцията `time(0)`, която дава текущото време на системния часовник в стандарт `epoch time`. Достатъчно е `srand()` да се изпълни веднъж за цялото изпълнение на програмата.

Чрез следния примерен код може да се генерира редица от 10 (практически) случайни числа, които са различни при всяко изпълнение на програмата.

```
srand (time(0));  
for (int i = 0; i < 10; i++)  
{  
    std::cout << rand () << std::endl;  
}
```

Стойностите на `rand()` са в интервала `[0..INT_MAX]`. Ако е нужно да генерирате стойности в друг интервал, например `[0..N]`, това може да стане

по формулата $\frac{\text{rand}()}{\text{INT_MAX}} \times N$ (трябва да избегнете целочисленото делене)!

3.26. Да се измери емпирично времето за изпълнение на алгоритъма за сортиране по метода на мехурчето. Да се начертае графика на зависимостта на времето за изпълнение от големината на масива. Всеки тест да е с наново генериран масив от случайни числа.

3.27. Да се въведе матрица от числа $A_{N \times M}$.

- Да се сортира всеки от редовете на матрицата
- Да се сортира всяка от колоните на матрицата

Така получените матрици да се отпечатаат на стандартния изход.

Bozosort е случайностен алгоритъм за сортиране на масиви. При този алгоритъм, на всяка стъпка се разменят две случайни числа от масива, след което се проверява дали масивът се е сортирал. Процесът продължава до сортиране на масива.

3.28. Да се реализира алгоритъма **Bozosort**. Да се измери емпирично времето му за изпълнение. *Внимание: тествайте с достатъчно малки масиви, тъй като този алгоритъм е изключително бавен.*

3.5 Низове II

3.29. Задача 3.55. [1] Дадена е квадратна таблица $A_{n \times n}$ ($1 \leq n \leq 30$) от низове, съдържащи думи с максимална дължина 6. Да се напише програма, която проверява дали изречението, получено след конкатенацията на думите от главния диагонал (започващо от горния ляв ъгъл) съвпада с изречението, получено след конкатенацията на думите от вторичния главен диагонал на A (започващо от долния ляв ъгъл).

3.30. Задача 3.56. [1] Дадена е квадратна таблица A от n -ти ред ($1 \leq n \leq 20$) от низове, съдържащи думи с максимална дължина 9. Да се напише програма, която намира и извежда на екрана изречението, получено след обхождане на A по спирала в посока на движението на часовниковата стрелка, започвайки от горния ляв ъгъл. Например ако матрицата A има вида:

$$\begin{pmatrix} a & b & c \\ d & e & f \\ g & h & i \end{pmatrix}$$

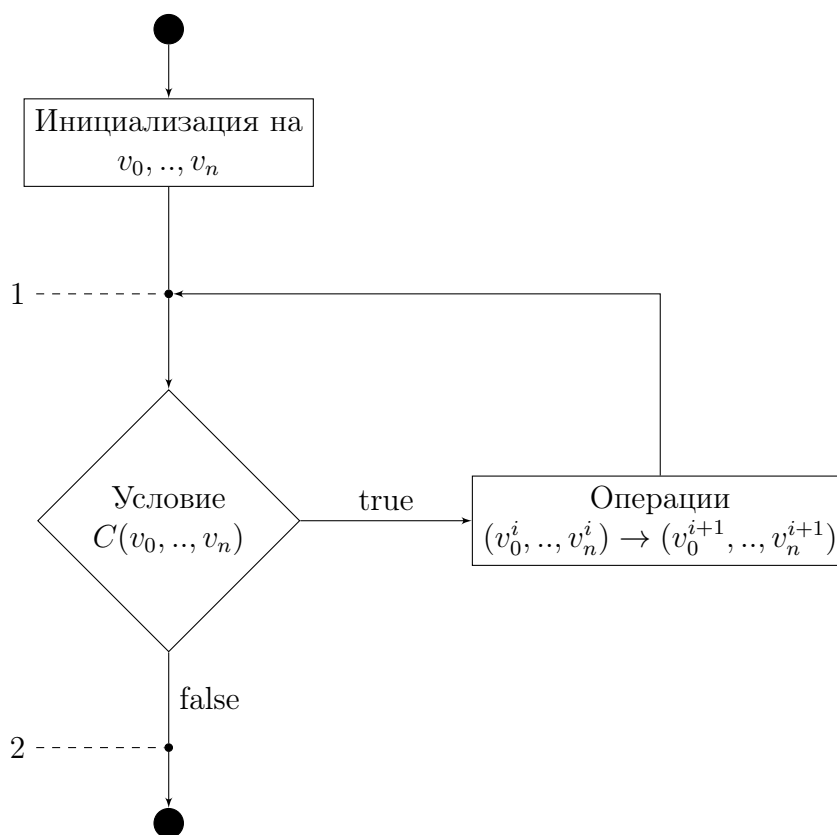
изречението след обхождането по спирала е: “abcfihgde”.

- 3.31. Задача 3.57 (Inner Join). [1] Нека са дадени два масива от низове – students и grades с най-много 20 низа във всеки. Низовете в масива students имат вида “XXXXXX YYYU...”, където “XXXXXX” е шестцифрен факултетен номер, а “YYYU...” е име с произволна дължина. Низовете в grades имат вида “XXXXXX YYYU”, където “XXXXXX” е шестцифрен факултетен номер, а “YYYU” е оценка под формата на число с плаваща запетая. И двата масива са сортирани във възходящ ред по факултетен номер. Възможно е в някой от двата масива да има данни за факултетни номера, за които няма данни в другия. И в двата списъка даден факултетен номер се среща най-много един път. Да се напише програма, която извежда на екрана имената и оценките на тези студенти, за които има информация и в двата списъка, като оценките са увеличени с 1 единица, но са максимум 6.00.
- 3.32. Даден е символен низ с най-много 255 произволни символа. “Дума” в низа наричаме всяка непразна последователност от символи, разделени от останалите с интервали или краищата на низа. Да се изведе списъка с всички различни думи, съдържащи се в дадения низ, без повторения.
- 3.33. Даден е символен низ с най-много 255 произволни символа. “Дума” в низа наричаме всяка непразна последователност от символи, разделени от останалите с интервали или краищата на низа. Да се изведе списъка с всички различни думи, съдържащи се в дадения низ, без повторения, като за всяка дума се посочи и броя на срещанията ѝ в низа.
- 3.34. Даден е символен низ с най-много 255 произволни символа. “Дума” в низа наричаме всяка непразна последователност от символи, разделени от останалите с интервали или краищата на низа. Да се изведе списъка с всички различни думи, съдържащи се в дадения низ, без повторения, като за всяка дума се посочи и броя на срещанията ѝ в низа. Списъкът да е сортиран по ред на броя на срещанията на думите.

3.6 Инвариант на цикъл и верификация на програми

Изложението в тази секция е силно опростена версия на метода на Флойд (Floyd) за верификация на блок схеми, описан в монументалната му статия [4]. Редица идеи и задачи са взаимствани от книгата [5] на А. Соскова и С. Николова “Теория на програмите в задачи”. За по-подробно изложение и допълнителни примери и задачи препоръчваме тази книга. Изложението по-долу е нарочно опростено, като на места е жертвана прецизността му.

Инвариант на цикъл

Фигура 1.6: Блок схема на част от програма с **while** цикъл

Нека е дадена програма или част от програма, която се състои от единствен **while** цикъл. Нека имаме някакъв набор от работни променливи $v_1, \dots, v_n$, които се инициализират непосредствено преди **while** цикъла. Условието на цикъла зависи изцяло от работните променливи, а тялото на цикъла използва или променя само тях. Приемаме, че програмата не произвежда странични ефекти и не зависи от такива.

Тоест, за програмата знаем, че: (1) не извършва входни и изходни операции, (2) поведението ѝ зависи изцяло от началните стойности на работните променливи $v_1, \dots, v_n$ и (3) не модифицира никакви други променливи, освен работните. Такава програма можем да изобразим чрез блок схемата на Фигура 1.6.

На фигурата $C(v_1, \dots, v_n)$ е някакъв логически израз, зависещ само от $v_1, \dots, v_n$. Да допуснем, че сме “замразили” изпълнението на програмата в точката, обозначена с “1”, точно преди да започне i -тото поред ($i \geq 0$) изпълнение на тялото на цикъла, т.е. непосредствено преди да бъдат изпълнени

операторите в него за i -ти път. В този момент n -торката работни променливи $(v_1, \dots, v_n)$ имат някакви конкретни стойности. Да ги обозначим с $(v_0^i, \dots, v_n^i)$. След изпълнение на всички оператори в тялото на цикъла работните променливи получават някакви нови стойности. Това са точно стойностите $(v_0^{i+1}, \dots, v_n^{i+1})$, които работните променливи ще имат в началото на $i + 1$ -вата итерация. Това е изобразено на фигурата чрез прехода $(v_0^i, \dots, v_n^i) \rightarrow (v_0^{i+1}, \dots, v_n^{i+1})$.

Нека приемем, че при някакво конкретно изпълнение на програмата, при конкретни дадени начални стойности на работните променливи $(v_0^0, \dots, v_n^0)$, тялото на цикъла ще бъде изпълнено точно $k \geq 0$ пъти, след което условието $C(v_1, \dots, v_n)$ на цикъла ще се наруши и той ще приключи. Изпълнението на програмата достига точката, обозначена с “2”. По този начин се получава редицата от n -торки $(v_0^0, \dots, v_n^0), \dots, (v_0^k, \dots, v_n^k)$, като за всички нейни членове $0 \leq i < k$, освен за последния, знаем че е вярно $C(v_0^i, \dots, v_n^i)$, а за последния, k -ти член, е вярно обратното — $\neg C(v_0^k, \dots, v_n^k)$.

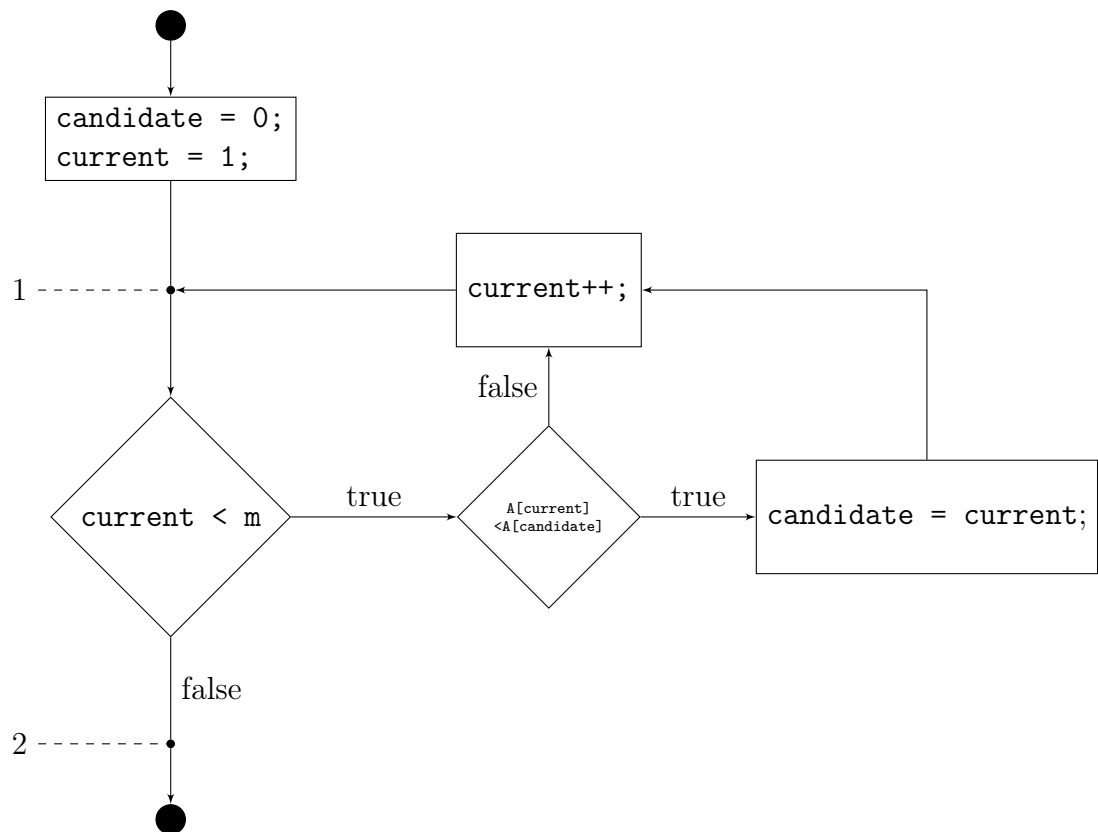
Освен свойството $C(v_0^i, \dots, v_n^i)$ (или неговото отрицание), което знаем за всички последователни стойности на работните променливи, можем да въведем още едно свойство $I(v_0^i, \dots, v_n^i)$, наречено “инвариант на цикъла”. За свойството I искаме да е вярно за всички възможни стойности на работните променливи, дори и за последния член на редицата им. От там идва името “инвариант”, т.е. факт, което е винаги верен.

Това свойство може да е произволно, например тавтологичната истина **true** изпълнява условието за инвариант, тъй като е вярна за всички членове на редицата на работните променливи. Инвариантът обаче е най-полезен, когато представя “смисъла” на работните променливи, и ако от верността на $\neg C(v_0^k, \dots, v_n^k) \& I(v_0^k, \dots, v_n^k)$ можем да изведем нещо полезно за “крайния резултат” от изпълнението на цикъла, съдържащ се в стойностите $(v_0^k, \dots, v_n^k)$.

Верификация на програми

Понятието “инвариант” ще илюстрираме чрез едно негово приложение: “Верификация на програма”. Верификацията на програма е доказателство, че при необходимите начални условия дадена програма изчислява стойност, удовлетворяваща някакво желано свойство. Като пример да разгледаме следната програма, за която ще се уверим, че намира най-малък елемент на едномерен масив A с $m > 0$ елемента $A[0], \dots, A[m - 1]$.

```
size_t candidate = 0, current = 1;
while (current < m)
{
```



Фигура 1.7: Блок схема на програмата за намиране на най-малък елемент в масив

```

if (A[candidate] < a[current])
{
    candidate = current;
}
current++;
}

```

Можем да приложим метода на инварианта, за да докажем строго, че когато цикълът приключи, то елементът $A[\text{candidate}]$ е гарантирано по-малък или равен на всички останали елементи на масива. Като първа стъпка, за улеснение изобразяваме програмата като блок схемата на Фигура 1.7.

Работните променливи на програмата, освен масива A , са целочислените променливи **candidate** и **current**. Какъв е смисълът на тези променливи? Веднага се вижда, че **current** е просто брояч — служи за обхождане на елементите на масива последователно от $A[0]$ до $A[m-1]$, като на всяка итерация от цикъла се разглежда стойността на $A[\text{current}]$.

Интуитивно се вижда, че **candidate** е намереният най-малък елемент на масива до текущия момент от обхождането. Тоест, можем да се надяваме, че ако сме разгледали първите i елемента на масива, то правилно сме определили, че най-малкият от тях е $A[\text{current}]$.

Тези размишления може да запишем чрез инварианта:

$$I ::= A[\text{candidate}] = \min(A[0], \dots, A[\text{current} - 1]).$$

Този инвариант очевидно е верен в началото на изпълнението на програмата, когато $\text{current} = 1$, а $\text{candidate} = 0$. Как да се уверим, че инвариантът е валиден за всички стойности, през които преминават работните променливи?

Нека допуснем, че инвариантът е верен за някаква стъпка i от изпълнението на цикъла. Тоест, допускаме $I(\text{candidate}^i, \text{current}^i)$, или $A[\text{candidate}^i] = \min(A[0], \dots, A[\text{current}^i - 1])$.

На итерация $i + 1$ имаме и $\text{current}^{i+1} = \text{current}^i + 1$, и следователно трябва да покажем, че $A[\text{candidate}^{i+1}] = \min(A[0], \dots, A[\text{current}^{i+1} - 1])$.

След навлизане в тялото на цикъла, имаме две възможности:

- 1) $A[\text{current}^i] \geq A[\text{candidate}^i]$. От допускането следва, че добавянето на $A[\text{current}^i]$ към $\min(A[0], \dots, A[\text{current}^i - 1])$ не променя стойността на минимума, или $\min(A[0], \dots, A[\text{current}^i - 1]) = \min(A[0], \dots, A[\text{current}^i - 1], A[\text{current}^i])$. От тук директно се вижда, че инвариантът е верен за итерация $i + 1$.
- 2) $A[\text{current}^i] < A[\text{candidate}^i]$. Тъй като по допускане $A[\text{candidate}^i] = \min(A[0], \dots, A[\text{current}^i - 1])$, от тук следва, че $A[\text{current}^i] < \min(A[0], \dots, A[\text{current}^i - 1])$, следователно $A[\text{current}^i] = \min(A[0], \dots, A[\text{current}^i - 1], A[\text{current}^i])$.

Но след присвояването имаме $candidate^{i+1} = current^i$, от където $A[candidate^{i+1}] = \min(A[0], \dots, A[current^i])$. Но $current^i = current^{i+1} - 1$, от където получаваме верността на I за итерация $i+1$: $A[candidate^{i+1}] = \min(A[0], \dots, A[current^{i+1} - 1])$.

Доказателството протича по индукция. Уверяваме се, че инвариантът е верен за цялата редица от стойности на **candidate** и **current**.

Какво се случва в края на цикъла, когато имаме $\neg(current < m)$? Тъй като числата са цели, а **current** се увеличава само с единица, можем да заключим, че $current = m$. Замествайки това равенство в инварианта, получаваме твърдението

$$A[candidate] = \min(A[0], \dots, A[m - 1]).$$

Тоест, намерили сме минималния елемент на масива.

3.35. Да се докаже строго, че за следната функция е вярно $pow(x, y) = x^y$:

```
a) unsigned int pow (unsigned int x, unsigned int y)
{
    unsigned int p = 1, i = 0;
    while (i < y)
    {
        p *= x;
        i++;
    }
    return p;
}

б) unsigned int pow (unsigned int x, unsigned int y)
{
    unsigned int p = 1;
    while (y > 0)
    {
        p *= x;
        y--;
    }
    return p;
}

в) unsigned int pow (unsigned int x, unsigned int y)
{
    unsigned int z = x, t = y, p = 1;
    while (t > 0)
    {
        if (t%2 == 0)
        {
            z *= z;
        }
    }
}
```

```
        t /= 2;
    }else{
        t = t - 1;
        p *= z;
    }
}
return p;
}
```

3.36. Да се докаже строго, че за следната функция е вярно $\text{sqrt}(n) = \lfloor \sqrt{n} \rfloor$.

```
unsigned int sqrt (unsigned int n)
{
    unsigned int x = 0, y = 1, s = 1;
    while (s <= n)
    {
        x++;
        y += 2;
        s += y;
    }
    return x;
}
```

3.37. Да се напише програма, която проверява дали дадени два масива A и B с еднакъв брой елементи m са еднакви, т.е. съдържат същите елементи в същия ред. Да се докаже строго, че програмата работи правилно.

3.38. Да се докаже, че следната функция връща истина тогава и само тогава, когато масивът A с n елемента съдържа елемента x .

```
bool member (int A[], size_t n, int x)
{
    size_t i = 0;
    while (i < n && A[i] != x)
    {
        i++;
    }
    return i < n;
}
```

3.39. Да се напише програма, която проверява дали даден масив A с m елемента е сортиран, т.е. дали елементите му са наредени в нарастващ ред. Да се докаже строго, че програмата работи правилно.

Съществуват редица съвременни методи за верификация на програми. Пълната версия на метода, използван по-горе, се нарича “логика на Флойд-Хоар” (Floyd–Hoare logic) и е само един представител на този клас методи.

Също така, в компютърните науки има направление, наречено “Синтез на програми” (Program refinement). При синтеза на програми се решава обратната задача: по спецификация на входа и изхода да се генерира програма, която удовлетворява спецификацията.

Препоръчваме на любознателния читател да се запознае с логиката на Floyd–Hoare и методите за синтез на програми.

4 Указатели и програмен стек

4.1 Предаване на масиви и указатели като параметри на функции

- 4.1. Да се дефинира функция, която получава като параметри два масива с еднакъв брой елементи. Функцията да разменя съответните елементи на масивите ($a[i] \leftrightarrow b[i]$).
- 4.2. Да се дефинира функция `swap([подходящ тип] a, [подходящ тип] b)`, която разменя стойностите на две целочислени променливи, предадени на функцията чрез `a` и `b`.

Каква е разликата с предишната задача? Защо при тази задача се налага използване на псевдоними?

4.2 Масиви от указатели

- 4.3. Да се напише булева функция `bool duplicates (long *pointers[])`, която получава като параметър масив `pointers` от указатели към целочислени променливи. Функцията да проверява дали поне две от съответните променливи имат еднакви стойности.
- 4.4. Да се дефинира функцията `bool commonel (int *arrays[], int npointers, int arlengths[])`. Масивът `arrays` съдържа `npointers` на брой указатели към масиви от цели числа. `i`-тият масив има големина `arlengths[i]`. Функцията да връща истина, ако има поне едно число x , което е елемент на всички масиви.
- 4.5. Да се дефинира функцията `bool subarrays (int *arrays[], int npointers, int arlengths[])`. Масивът `arrays` съдържа `npointers` на брой указатели към масиви от цели числа. `i`-тият масив има големина `arlengths[i]`. Функцията да връща истина, ако поне един от масивите е подмасив на друг масив. Масивът a наричаме подмасив на b , ако заетата от a памет е част от заетата от b памет. Да се напишат подходящи тестове за функцията.

4.3 Програмен стек

- 4.6. Да се дефинира рекурсивна функция `double sum(size_t n)`, която въвежда n числа от стандартния вход връща сумата им. *Да не се използват оператори за цикъл!*
- 4.7. Да се дефинира рекурсивна функция `reverse(n)`, която въвежда n числа от стандартния вход и ги извежда в обратен ред. *Да не се използват масиви. Да се използва програмния стек чрез рекурсия.*

- 4.8. Да се дефинира функция `void getmax (long *pmax, size_t n)`, която въвежда n числа от стандартния вход и записва максималното от тях в променливата, сочена от указателя `pmax`.

Пример: Следната програма ще изведе най-малкото от 5 въведени от стандартния вход числа.

```
int main ()
{
    long max = -1;
    getmax (&max,5);
    std::cout << max;

    return 0;
}
```

Функцията да се реализира по два начина: чрез цикъл и чрез използване на рекурсия без оператори за цикъл.

5 Рекурсия

5.1 Прости рекурсивни функции

- 5.1. Задача 5.2.[1] Да се дефинира рекурсивна функция за намиране на стойността на полинома на Ермит $H_n(x)$ (x е реална променлива, а n неотрицателна цяла променлива), дефиниран по следния начин:

$$H_0(x) = 1$$

$$H_1(x) = 2x$$

$$H_n(x) = 2xH_{n-1}(x) + 2(n-1)H_{n-2}(x), n > 1$$

- 5.2. Задача 5.3.[1] Произведението на две положителни цели числа може да се дефинира по следния начин:

$$\text{mult}(m, n) = m, \text{ ако } n = 1$$

$$\text{mult}(m, n) = m + \text{mult}(m, n-1), \text{ иначе.}$$

Да се дефинира рекурсивна функция, която намира произведението на две положителни цели числа по описания по-горе начин.

- 5.3. Задача 5.5.[1] Да се дефинира функция, която намира най-големия общ делител на две неотрицателни цели числа, поне едното от които е различно от 0.

- 5.4. Задача 5.7.[1] Дадени са естествените числа n и k ($n \geq 1, k > 1$). Да се дефинира рекурсивна функция, която намира произведението на естествените числа от 1 до n със стъпка k .

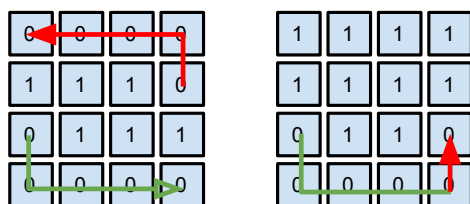
- 5.5. Задача 5.10.[1] Дадено е неотрицателно цяло число n в десетична бройна система. Да се дефинира рекурсивна функция, която намира сумата от цифрите на n в бройна система с основа k ($k > 1$).

- 5.6. Задача 5.11.[1] Да се дефинира рекурсивна функция, която установява дали в запис на неотрицателното цяло число n , записано в десетична бройна система, се съдържа цифрата k .

- 5.7. Задача 5.19.[1] Да се дефинира рекурсивна функция, която проверява дали дадено положително цяло число е елемент на редицата на Фибоначи.

- 5.8. Задача 5.28.[1] Да се дефинира рекурсивна функция, която намира максималния елемент на редицата от цели числа $a_0, a_1, a_2, \dots, a_{n-1}$, където $n \geq 1$.

Забележка: Редицата е представена като масив.



Фигура 1.8: Примерени лабиринти

- 5.9. Задача 5.31.[1] Да се напише функция

```
void insertSorted (long x, long arr[], long n),
```

която включва цялото число x число в сортирана във възходящ ред редица от цели числа arr , в която има записани n елемента. Вмъкването да запазва наредбата на елементите. Предполага се, че за редицата е заделена достатъчно памет за допълване с още едно число.

- 5.10. Задача 5.34.[1] Да се дефинира рекурсивна функция, която сравнява лексикографски два символни низа.

5.2 Търсене с пълно изчерпване

- 5.11. Нека е дадена квадратна матрица от цели числа $N \times N$, представляваща “лабиринт”. Елементи на матрицата със стойност 0 смятаме за “проходими”, а всички останали - за “непроходими”. Път в лабиринта наричаме всяка последователност от проходими елементи на матрицата, които са съседни вертикално или хоризонтално, такава че (1) никой елемент от последователността не е последван директно от предшественика си (забранено е “връщането назад”) и (2) най-много един елемент на последователността се среща в нея повече от веднъж (има най-много един “цикъл”).

Да се дефинира функция `bool downstairs (int sx, int sy, int tx, int ty)`, която проверява дали съществува път от елемента (sx, sy) до елемента (tx, ty) , такъв, че всеки следващ елемент от пътя е или вдясно, или под предишния. Такъв път да наричаме “низходящ”.

Пример: На Фигура 1.8(a) такъв път съществува от елемента $(0, 2)$ до елемента $(3, 3)$, но не и от $(3, 1)$ до $(0, 0)$.

- 5.12. При условията на дефинициите от предишната задача, да се дефинира функция `bool connected()`, която проверява дали от всеки елемент на

матрицата (sx, sy) до всеки елемент на матрицата (tx, ty) , такива, че $sx \leq tx$ и $sy \leq ty$, съществува низходящ път.

Пример: За лабиринта от Фигура 1.8(а) условието е изпълнено, но не и за лабиринта от Фигура 1.8(б).

- 5.13. Да се напише програма, която по въведени от клавиатурата $4 \leq n \leq 8$ и $0 \leq k \leq n$ намира извежда на екрана всички възможни конфигурации на абстрактна шахматна дъска с размери $n \times n$ с разположени на нея k коня така, че никоя фигура не е поставена на поле, което се “бие” от друга фигура според съответните шахматни правила.

Пример за отпечатана конфигурация с $n = 5, k = 2$:

```

- - - - -
- - Н - -
- - - - -
- - - Н -
- - - - -

```

- 5.14. При условията на задача 5.11. да се напише функция

```
int minDistance (int sx, int sy, int tx, int ty),
```

която по въведени от клавиатурата координати на елементи $s = (sx, sy)$ и $t = (tx, ty)$ намира *дължината* на най-краткия път между s и t . Обърнете внимание, че се иска *път*, а не просто низходящ път.

- 5.15. При условията на задача 5.11. да се напише функция, която по въведени от клавиатурата координати на елементи $s = (sx, sy)$ и $t = (tx, ty)$ намира и отпечатва на екрана елементите, от които се състои най-краткия път между s и t . Обърнете внимание, че се иска *път*, а не просто низходящ път.

- 5.16. Пъзел на Синди[6].

Дадена е игрова дъска като на фигура 2, която се състои от n черни и n бели фигури. Фигурите могат да бъдат разположени на $2n + 1$ различни позиции. Играта започва с разполагане на всички черни фигури вляво, а всички бели - вдясно на дъската.

Черните фигури могат да се местят само надясно, а белите - само наляво. На всеки ход важат следните правила:

- всяка фигура се мести само с по една позиция, ако съответната позиция не е заета;

- ако позицията е заета, фигурата X може да прескочи точно една фигура Y от противоположния цвят, ако позицията след Y е свободна.

Да се напише програма, която по въведено число n отпечатва на екрана инструкции за игра така, че в края на играта всички бели фигури да са вляво на дъската, а всички черни - вдясно. Инструкциите да са от следния вид:

...

Преместете черна фигура от позиция 1 на позиция 2.

Преместете бяла фигура от позиция 5 на позиция 3.

...

Допустимо е инструкциите да бъдат отпечатани в обратен ред.

На следните фигури е даден пример за игра:



1. Начална конфигурация.



2. Преместване на черна фигура с един ход надясно.



3. Преместване на бяла фигура с прескачане.



4. Преместване на черна фигура с един ход надясно.



5. Преместване на черна фигура чрез прескачане

O	X
X	O

Фигура 1.9: Невъзможна игра

След ход 5 конфигурацията на играта е безперспективна.

- 5.17. Да се генерират всички възможни коректни конфигурации, които могат да се получават при играта Морски шах с размер на игровото поле $N \times N$ ($N = 2$ или 3). *Внимание! Фигура 1.9 илюстрира невъзможна игра. Играта трябва да е приключила преди да се постави второто 'O', ако приемем, че първият играч е 'X'.*
- 5.18. Да се напише програма, която извежда всички възможни редици, които могат да се образуват от първите N естествени числа (техните пермутации). Например, за $N=2$, това са редиците $\{0, 1\}$ и $\{1, 0\}$.

Някои от задачите по обектно-ориентирано програмиране са решени в сборника [7] *Магдалина Тодорова, Петър Армянов, Калин Николов, “Сборник от задачи по програмиране на C++. Част втора. Обектно-ориентирано програмиране”*. За тези задачи е запазена номерацията в сборника.

6 Структури

Задачите за полиноми са на базата на разработените на лекции примери за полиноми, представени чрез структурата:

```
const size_t maxPower = 50;
struct Poly
{
    double coefs[maxPower];
    size_t power;
};
```

- 6.1. Да се реализира функцията `diff` за намиране на първата производна на полином относно променливата `x`.

Задачата да се реши в два варианта: като функция изображение $Poly \rightarrow Poly$ и като “деструктивна” функция с тип на резултата `void`, която изменя стойността на аргумента си.

И двете функции да се тестват с подходящи примери!

- 6.2. Да се реализира функцията `prod` за умножение на два полинома.

- 6.3. Задача 1.1. [7] Нека е дефинирана структурата `Product`:

```
struct Product
{
    char description[32];
    //описание на изделие
    int partNum;
    //номер на изделие
    double cost;
    //цена на изделие
};
```

- а) Да се създадат две изделия и се инициализират чрез следните данни:

description	partNum	cost
screw driver	456	5.50
hammer	324	8.2-0

- б) Да се изведат на екрана компонентите на двете изделия;
 в) Да се дефинира масив от 5 структури `Product`. Елементите на масива да не се инициализират;

г) Да се реализира цикъл, който инициализира масива чрез нулевите за съответния тип на полетата стойности;

д) Да се променят елементите на масива така, че да съдържат следните стойности:

description	partNum	cost
screw driver	456	5.50
hammer	324	8.20
socket	777	6.80
plier	123	10.80
hand-saw	555	12.80

е) Да се изведат елементите на масива на конзолата с подходящо форматиране;

6.4. Задача 1.4. [7] Да се дефинират структурите `polar` и `rect`, задаващи вектор с полярни и с правоъгълни координати съответно. Да се дефинират функции, които преобразуват вектор, зададен чрез правоъгълни координати, в полярни координати и обратно, както и функции, които извеждат вектор, зададен чрез полярните си и чрез правоъгълните си координати.

В главната функция да се дава възможност за избор на режим на въвеждане: `r` – за въвеждане в правоъгълни и `p` – в полярни координати. За всеки избран режим да се въведат произволен брой вектори, да се преобразуват в другия режим и да се изведат.

6.5. Задача 1.8.[7] Да се дефинира функция, която сортира лексикографски във възходящ ред редица от точки в равнината. За целта да се дефинира структура `Point`, описваща точка от равнината с декартови координати.

6.6. Задача 1.Б.5.[7] Да се дефинира структура `Planet`, определяща планета по име (символен низ), разстояние от слънцето, диаметър и маса (реални числа). Да се дефинират функции, изпълняващи следните действия:

а) въвежда данни за планета от клавиатурата;

б) извежда данните за планета;

в) връща като резултат броя секунди, които са необходими на светлината да достигне от слънцето до планетата (да се приеме, че светлината има скорост 299792 km/s и че разстоянието на планетата до слънцето е зададено в километри).

г) създава едномерен масив от планети с фиксиран размер и въвежда данните за тях от стандартния вход;

д) извежда данните за планетите от масив, подаден на функцията като параметър;

- е) отпечатва данните за планетата с най-голям диаметър от масив, подаден на функцията като параметър;

7 Динамична памет

7.1 Заделяне на динамична памет

- 7.1. Задача 1.4.24. [1] Да се дефинира функция `strduplicate`, която създава копие на символен низ. Функцията да се грижи за заделянето на памет за новия низ.
- 7.2. Задача 1.4.25. [1] Да се дефинира функция, която преобразува положително цяло число в съответния му символен низ и връща така построения символен низ.
- 7.3. Задача 1.4.30. [1] Обединение на два символни низа s_1 и s_2 наричаме всеки символен низ, който съдържа без повторение всички символи на s_1 и s_2 . Да се дефинира функция, която намира и връща обединението на два символни низа.
- 7.4. Задача 1.4.36. За [1] работа със символни низове могат да бъдат използвани следните основни функции:
- `char car(const char* x)`, която връща първия символ (елемент) на низа x ;
 - `char* cdr(char* x)`, която връща останалата част от низа x след отделянето на първия елемент на низа x ;
 - `char* cons(char x, const char* y)`, която връща указател към символен низ, разположен в динамичната памет и съдържащ конкатенацията на символа x със символния низ y ;
 - `bool eq(const char* x, const char* y)`, която връща `true` тогава и само тогава, когато низовете съвпадат.

Да се дефинират описаните функции. Като се използват тези функции, да се дефинират следните функции:

- `char* reverse(char* x)`, която връща указател към символен низ, разположен в динамичната памет и съдържащ символите на x , записани в обратен ред;
- `char* copy(char* x)`, която връща указател към символен низ, разположен в динамичната памет и съдържащ копие на символния низ x ;
- `char* car_n(char* x, int n)`, която връща указател към символен низ, разположен в динамичната памет и съдържащ първите n символа на символния низ x ;

- `char* cdr_n(char* x, int n)`, която връща останалата част от низа `x` след отделянето на първите `n` символа. Предварително е известно, че `x` притежава поне `n` символа;
- `int number_of_char(char* x, char ch)`, която намира колко пъти символът `ch` се среща в символния низ `x`;
- `int number_of_substr(char* x, char* y)`, която намира колко пъти символният низ `y` се среща в символния низ `x`;
- `char* delete_substr(char* x, char* y)`, която връща указател към символен низ, разположен в динамичната памет и съдържащ символите на низа `x`, от който са изтрети всички срещания на символния низ `y`.

7.2 Масиви от структури в динамичната памет и текстови файлове

- 7.5. Решението на задача 6.3. да се разшири така, че масив от структури да може да се протича от и записва във текстов файл. Броят на записаните във файла структури да може да е произволен и да се използва динамична памет за инициализация на масива с необходимия брой елементи.
- 7.6. Решението на задача 6.5. да се разшири като се добави диалогов режим (т.нар. “меню”), чрез който може да се модифицира съдържанието на глобална редица (масив) от точки в равнината:
- а) Да може към редицата от точки да се добавят $n \geq 2$ точки от отсечка с начало точката (x_1, y_1) и край точката (x_2, y_2) . Координатите на началото и края, както и числото n , се задават от потребителя. Точките да са на равно разстояние помежду си.
 - б) Да могат да се изпишат на стандартния изход всички точки от редицата, които са в посочен от потребителя кавадрант.
 - в) Да може да се премахнат всички точки, лежащи на дадена права. Правата се въвежда чрез коефициентите на уравнението на правата $ax + by + c = 0$.
 - г) Редицата от точки да може да се запише в текстов файл “points.dat”.
 - д) Редицата от точки да може да се прочете от текстов файл “points.dat”. Ако текущата работна редицата не е празна, съществуващите точки се изтриват.

7.3 Динамични масиви

Често се налага функции да създават масиви в динамичната памет и да ги връщат като резултат. Както е известно, за да може такъв масив да се използва нормално, трябва да е известен не само адреса на първия му елемент, но и броя на елементите му. Поради това, функциите, заделящи масиви, е необходимо да връщат два резултата. С прости средства това може да се постигне, например, ако размера на масива се връща чрез параметър от тип “псевдоним”:

```
int* createIntArray (size_t &array_size,...){...}
```

Вместо това, можем да обединим адреса на буфера и броя на елементите на масива в структура:

```
struct intarray
{
    int *arr;
    size_t size;
};
```

По този начин `createIntArray` би придобила формата:

```
intarray createIntArray (...){...}
```

Функции, които работят с няколко масива, също се дефинират по-елегантно. Например:

```
intarray append (intarray a, intarray b){...}
```

Горната функция би изглеждала значително по-сложно, ако трябваше всеки масив да представяме с две величини.

7.7. Да се дефинира функция

```
intarray append (intarray a, intarray b),
```

създаваща нов масив в динамичната памет, съдържащ последователно елементите на `a` и `b`.

7.8. Да се дефинира функция `equals(a,b)`, която проверява дали два динамична масива са еднакви (съдържат еднакви елементи в еднакъв ред).

7.9. Да се дефинира функция `assign(a,b)`, която присвоява масива `b` на масива `a`, аналогично на операцията `a=b` за простите типове в C++.

7.10. Да се дефинира функция

```
intarray mergesorted (intarray a, intarray b).
```

a и **b** са подредени в нарастващ ред. Функцията да създава нов масив в динамичната памет, съдържащ всички елементи на **a** и **b** също в нарастващ ред. Например, резултатът от сливането на `[1,3,3,5]` и `[2,4,5]` би бил `[1,2,3,3,4,5,5]`. Функцията да обхожда елементите на **a** и **b** точно по веднъж (да работи с линейна сложност).

7.11. Да се дефинира функция

```
intarray union (intarray a, intarray b),
```

създаваща нов масив в динамичната памет, съдържащ всички елементи на **a** и **b** без повторения (обединение на множествата на елементите).

7.12. Да се дефинира функция

```
intarray intersection (intarray a, intarray b),
```

създаваща нов масив в динамичната памет, съдържащ елементите, които се съдържат едновременно в **a** и **b** (сечение на множествата на елементите).

7.13. Да се дефинира функция

```
intarray evens (intarray a),
```

създаваща нов масив в динамичната памет, съдържащ само тези елементи на **a**, които са четни числа.

8 Шаблиони и указатели към функции

8.1 Шаблиони на функции

- 8.1. Да се реализира шаблон на функция `void input ([подходящ тип] array, int n)`, която въвежда от клавиатурата стойностите на елементите на масива `array` от произволен тип `T` с големина `n`.

Какви са допустимите типове T за този шаблон? Защо функцията е от тип `void`?

Да се реализира и изпълни подходящ тест за функцията.

- 8.2. Да се реализира шаблон на функция `bool ordered ([подходящ тип] array, int n)`, която проверява дали елементите на масива `array` от произволен тип `T` с големина `n` образуват монотонно-растяща редица спрямо релацията `<`.

Какви са допустимите типове T за този шаблон?

Да се реализира и изпълни подходящ тест за функцията.

- 8.3. Да се реализира шаблон на функция `bool member ([подходящ тип] array, int n, [подходящ тип] x)`, която проверява дали `x` е елемент на масива `array` от произволен тип `T` с големина `n`.

Има ли в `C++` тип T , който не е съвместим с този шаблон?

Да се реализира и изпълни подходящ тест за функцията.

8.2 Функции от високо ниво

- 8.4. Да се дефинира масив `functions` с 5 елемента от тип функция $double \rightarrow double$. Да се дефинират 5 произволни функции от този тип и адресите им да се присвоят на елементите на масива.

При въведено от клавиатурата число $x : double$, да се намери и отпечата индексът на тази функция в масива `functions`, чиято стойност е най-голяма в точката x спрямо стойностите на всички функции в масива. Ако има няколко такива функции, да се отпечата индекса на коя да е от тях.

- 8.5. Да се дефинира функция `double fmax([подходящ тип]f, [подходящ тип]g, double x)`, където f и g са две произволни функции от тип $double \rightarrow double$, за които приемаме, че са дефинирани в x . Функцията да връща по-голямата измежду стойностите на f и g в точката x .

Да се реализира и изпълни подходящ тест за функцията.

- 8.6. Да се дефинира функция `double maxarray([подходящ тип] array, int n, double x)`, където `array` е масив от функции от тип $double \rightarrow double$ с големина n .

Функцията `maxarray` да връща най-голямата измежду стойностите на всички функции от масива в точката x като приемаме, че всички те са дефинирани в тази точка.

Задачата да се реши със и без използването на функцията `fmax` от предишната задача.

Да се реализира и изпълни подходящ тест за функцията.

- 8.7. Нека е дадена следната структура: `struct S {int a; int b; int c;};`. Да се дефинира функция `void sort([подходящ тип]array, int n, [подходящ тип]compare)`, където `array` е масив от n структури от тип `S`.

Типът на функцията `compare` да се подбере така, че чрез нея да може да се реализира произволна наредба за типа `S`, т.е. функцията да може да сравнява “по големина” две структури от `S` по произволен критерий.

Да се създаде и инициализира масив с 5 структури от тип `S`. Като се използва функцията `sort` да се сортира масива по веднъж по всеки от следните начини:

- а) по полето **a**
- б) по полето **b**
- в) лексикографски по тройката (a, b, c)

Глава 2

Обектно ориентирано програмиране

9 Класове I: Прости класове, методи и оператори

9.1. Задача 2.39.[7] Да се дефинира клас `Time`, който определя момент от денонощието по зададени час и минути. Класът да съдържа подходящи методи за:

- достъп и промяна на часа и минутите с проверки за коректност;
- добавящ към времето цяло число минути;
- достъп до броя минути, изминали от началото на денонощието;
- оператор за сравнение (казваме, че $t_1 < t_2$, ако t_2 е по-късно в денонощието от t_1).

Да се предефинират операторите `+`, `-` и `*`, така че да могат да се събират и изваждат две времена, както и да се умножават време с цяло число и цяло число с време. Да се включи дефинираният клас в програмата и направят обръщения към член-функциите му и предефинираните оператори.

9.2. Да се дефинира структура `Point`, описваща точка в евклидовата равнина и клас `Line`, описващ права в евклидовата равнина, зададена чрез две нейни точки.

Класът `Line` да съдържа методи, чрез които може да се извършват следните операции:

- Проверка дали две прави са успоредни;

- Проверка дали дадена точка лежи на дадена права;
- Намиране на пресечната точка на две прави. Приемаме, че правите не са успоредни. Стойността на резултата може да е произволна в противен случай.
- Създаване на права, която е ъглополовяща на по-големия ъгъл, образуван от две прави. Стойността на резултата може да е произволна в противен случай.

Където е подходящо да се дефинират оператори вместо методи.

9.3. Задача 2.44. [7] (асоциативен масив) Да се дефинира клас `Dictionary`, който създава тълковен речник. Тълковният речник се състои от не повече от 500 двойки дума–тълкуване, като думата е символен низ с не повече от 100 символа, а тълкуването е символен низ с не повече от 500 символа.

- Да се дефинира подходяща структура, описваща една двойка дума–тълкуване;
- Да се дефинират подходящи член-данни на клас `Dictionary`;

Клас `Dictionary` да съдържа методи, с които може да се извършват следните операции над речника:

- Инициализация на празен речник;
- извеждане на всички думи в речника и техните тълкувания;
- включване на нова двойка дума–тълкуване в речника;
- изключване на двойка дума–тълкуване от речника (по дадена дума);
- търсене на значението на дадена дума в речник.
- извеждане на всички думи в речника и техните тълкувания по азбучен ред на думите;

Да се дефинира оператор `+`, обединяващ два речника, такъв че:

- Ако някои думи имат значение и в двата речника, значенията да се конкатенират в резултатния сумарен речник;
- Ако общият брой на думите в двата речника надхвърля 500, да се използват само първите 500 думи (при произволна наредба).

10 Жизнен цикъл на обектите

10.1 Конструктори

10.1. Да се дефинира клас `Word`, описващ дума, съставена от не повече от 20 символа от тип `char`. Класът да съдържа следните операции:

- оператор `[]` за намиране на *i*-тия пореден символ в думата
- оператори `+` и `+=` за добавяне на един символ в края на думата. Ако думата вече има 20 символа, операторите да нямат ефект
- оператори `<` и `==` за сравнение на думи спрямо лексикографската наредба
- подходящи конструктори

Да се реализира и изпълни подходящ тест за класа и неговите методи.

10.2. Да се реализира клас `NumbersSummator`, който поддържа сума на цели числа. При създаване на обект от класа, съответната му сума да се инициализира с число, което се подава като аргумент на конструктора. За класа да се реализират следните методи:

- `sum`, който връща текущата стойност на сумата
- `add`, увеличаващ сумата с дадено число
- `sub`, намаляващ сумата с дадено число
- `num`, връща колко пъти сумата е била променяна
- `average`, връщащ средното аритметично на всички числа, с които сумата е била променяна.

Забележка: Функционалност извън тези 4 метода, като например съхраняване на отделните числа от поредицата, не е необходима. Пример:

```
NumbersSummator seq1 (10);  
seq1.add (10);  
seq1.add (5);  
seq1.sub (15);  
cout << seq1.sum() ; //→10 (10+10+5-15)  
cout << seq1.average(); //→0 (10+5-15)/3
```

10.3. Да се дефинира клас `BrowserHistory`, който съдържа информация за историята на посещенията до най-много *N* Web сайта. *N* е параметър на конструктора на класа. За целта да се реализира структура `HistoryEntry`, описваща едно посещение на сайт чрез:

- a) Месец от годината, през който е посетен сайтът;
- б) Неговото URL.

Класът `codeBrowserHistory` да поддържа следните операции:

- Метод за добавяне на нов сайт към историята. Информацията за всеки сайт се въвежда от клавиатурата
- Оператор `+=` с параметър `HistoryEntry`, добавящ сайт към историята
- Метод за отпечатване на информацията за всички сайтове в историята
- Метод, който по даден месец от годината намира броя на сайтовете, посетени през този месец
- Намиране на този месец от годината, в който има най-много посетени сайтове
- Премахване на най-скоро добавеният сайт в историята
- Оператор `+`, който обединява двете истории

Да се реализира и изпълни подходящ тест за класа и неговите методи.

10.2 Управление на динамична памет

- 10.4. За клас `BrowserHistory` от задача 10.3. да се реализират конструктор за копиране, оператор за присвояване, оператори за събиране `+` и `+=`, обединяващи две истории и деструктор.

Да се реализира подходящ тест на класа.

- 10.5. Задача 2.2.44. (асоциативен масив) [7] Да се дефинира клас `Dictionary`, който създава тълковен речник. Тълковният речник се състои от не повече от 500 двойки дума-тълкуване, като думата е символен низ с не повече от 100 символа, а тълкуването е символен низ с не повече от 500 символа.

- Да се дефинира подходяща структура, описваща една двойка дума-тълкуване;
- Да се дефинират подходящи член-данни на клас `Dictionary`;

Клас `Dictionary` да съдържа методи, с които може да се извършват следните операции над речника:

- Инициализация на празен речник;
- извеждане на всички думи в речника и техните тълкувания;
- включване на нова двойка дума-тълкуване в речника;
- изключване на двойка дума-тълкуване от речника (по дадена дума);
- търсене на значението на дадена дума в речник.
- извеждане на всички думи в речника и техните тълкувания по азбучен ред на думите;

Да се дефинира оператор `+`, обединяващ два речника, такъв че:

- Ако някои думи имат значение и в двата речника, значенията да се конкатенират в резултатния сумарен речник;
- Ако общият брой на думите в двата речника надхвърля 500, да се използват само първите 500 думи (при произволна наредба).

- 10.6. Клас `Dictionary` от задача 10.5. да се реализира така, че максималният брой `N` на двойки ключ-стойност, които могат да бъдат добавени към речника, да се задава като параметър на конструктора на класа. За класа да се реализират конструктор за копиране, оператор за присвояване

и деструктор.

Да се реализират оператори за събиране $+$ и $+=$, обединяващи два речника. Ако в речниците **a** и **b** има еднакви думи с различни значения, то за тези думи в речника **a+b** да се използва значението им от речника **a**.

Да се реализира подходящ тест на класа.

Някои от следващите задачи са върху примерния шаблон **Vector**, работен на лекции. Шаблонът реализира прост контейнер чрез масив в динамичната памет:

```
template <typename T>
class Vector
{
public:
    Vector();
    Vector(const Vector<T> &);
    Vector<T>& operator=(const Vector<T> &);
    ~Vector ();
    Vector<T> operator + (const Vector<T> &) const;
    Vector<T>& operator+=(const Vector<T> &);
    T& operator[] (size_t i);
    T operator[] (size_t i) const;
    void push (const T& x);
    void print () const;
    size_t size() const;
private:
    T* elements;
    size_t nCapacity;
};
```

- 10.7. За шаблона **Vector** от лекции да се дефинира метод **Vector::resize**, с който да може динамично да се променя капацитета на масива. При намаляване на капацитета да отпаднат най-левите елементи на масива. При увеличаване на капацитета на масива, новите елементи да остават неинициализирани.

Да се реализират подходящи тестове.

- 10.8. Като се използва шаблона **Vector** да се създаде масив **M** то **3** елемента, чиито елементи са масиви от по **3** числа от тип **double**. Да се въведат елементите на **M** от клавиатурата.

10.9. За шаблона на клас `Vector` от лекции да се дефинира метод:

```
Vector<Vector<T>> Vector<T>::slice(size_t n).
```

Ако приемем, че изходният масив е с елементи от тип `T`, то методът `slice` създава и връща масив от масиви, т.е. резултатният масив се състои от масиви с елементи от тип `T`.

Методът да “разделя” изходният масив на равни по големина части с по `n` последователни елемента. `i`-тият поред масив от резултата съдържа `i`-тата поредна `n`-торка от последователни членове на изходния масив. Последният масив в резултата може да съдържа по-малко от `n` елемента, ако броят на елементите на изходният масив не е кратен на `n`.

Пример: Нека масивът `a` има елементите `[1,2,3,4,5,6,7,8,10,11]`. При тези условия, `a.slice(3)` създава и връща масива от масиви `[[1,2,3],[4,5,6],[7,8,9],[10,11]]`.

Да се напишат подходящи тестове.

10.10. Реализирайте шаблон на клас `Set`, като подберете подходящо представяне и методи за боравене с множество от елементи от тип `T`. Да се поддържат стандартни операции като:

- Добавяне и премахване на елементи от множеството
- Търсене на елемент в множеството
- Обединение, сечение, разлика на множества (като оператори)
- Оператори за сравнение (съвпадение, подмножество)

Да се напишат подходящи тестове. Да се дефинира оператор « за извеждане на съдържанието на множеството в поток.

11 λ функции

11.1 Map и Reduce

11.1. Да се изведат всички елементи на масив чрез функцията `map`.

11.2. Нека е дадена следната структура `struct S {int a; int b; int c;}`. Да се дефинира и попълни примерен масив `A` с елементи от `S`.

- а) Чрез подходяща помощна функция и използване на `map`, да се отпечата сумата на полетата `a`, `b` и `c` на всеки от елементите на `A`.
- б) Чрез подходяща помощна функция и използване на `map`, да се въведат елементите на `A`.
- в) Чрез подходяща помощна функция и използване на `map`, да се увеличи с единица всяко поле `a` на елементите на `A`.
- г) Чрез подходяща помощна функция и използване на `map`, да се разменят стойностите на полетата `a` и `b` на елементите на `A`.
- д) Да се тестват решенията на горните задачи.

За някои от следните задачи може да се приложи опростена форма на функцията `reduce`, с оператор $op : T \times T \rightarrow T$, при който съвпадат типовете на елементите, които се обработват, на акумулатора и на резултата. При числови задачи (сумирания, максимуми и т.н.) това е достатъчно.

В общия случай на приложението на `reduce` е възможно типа на резултата да е различен. Например, ако имаме масив от `double` числа и искаме да видим броя на положителните елементи, тогава операцията би била $op(el, accum) = accum + (el > 0)$. Тази операция следва да е от тип, например, $op : double \times int \rightarrow int$, тъй като броячът е цяло число. За целта е нужно да се дефинира `reduce` така, че типовете на елементите и на резултата да са различни. Например:

```
template <typename ElemT, ResultT>
using OpFn = std::function<ResultT (ElemT, ResultT)>

template <typename ElemT, ResultT>
ResultT reduce(ElemT arr[], unsigned size, OpFn,
               ResultT null_val);
```

11.3. Нека е дадена следната структура `struct S {int a; int b; int c;}`. Да се дефинира и попълни примерен масив `A` с елементи от `S` и чрез еднократно приложение на функцията `reduce` с подходяща операция, да се решат следните задачи:

- а) Да се намери обект **S res**, за който **res.a** е сумата на полето **a** на всички елементи на масива, **res.b** е сумата на полето **b**, а **c** - на полето **c**
 - б) Да се намери сумата на всички стойности в полето **a**
 - в) Да се намери броя на елементите на масива, за които **a==b==c**
 - г) Да се провери (т.е. да се генерира булев резултат) дали има поне една структура, за която **a >= b >= c**
- 11.4. (*) За даден масив от числа (**int**) $arr = \{a_1, \dots, a_n\}$ и функция $p : int \rightarrow int$, чрез еднократно приложение на **reduce** да се “разбие” списъка **l** на подсписъци, за елементите на които p има еднаква стойност. Т.е. да се построи масив от масиви от числа $G = \{G_1, \dots, G_k\}$, като елементите на G са съставени от елементите на arr и за и за всяко G_i , $p(x) == p(y)$ за всички $x, y \in G_i$. Операцията е сходна с операцията **Group by** известна от базите данни, която групира данните спрямо някакъв техен атрибут.
- Пример:* за масива $arr = \{1, \dots, 10\}$ и функцията $p(x) = x \% 3$, ще получим групите $\{\{3, 6, 9\}, \{1, 7, 10\}, \{2, 8\}\}$.
- Упътване:* за решаване на тази задача е най-практично да се използва вариант на **reduce** за динамичен масив.

11.2 λ функции и `std::function`

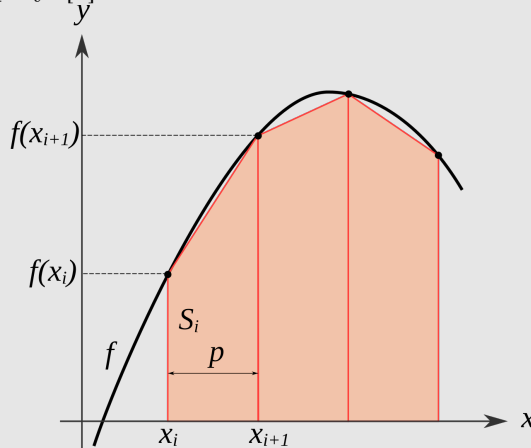
Някои от следните задачи са взаймствани от библиотеката на JavaScript за функционално програмиране “**lodash**”. За примери и повече информация: документация на библиотеката.

В тази секция, “функция” може да се разбира като “извикваем обект”. Т.е. не е задължително да използвате **lambda** функции за всички решения. Там, където ви се струва по-естествено, можете да дефинирате клас или структура с **operator()**. Полезно упражнение би било да опитате да решите задачите и по двата начина – с **lambda** функции и с “извикваеми” класове/структури.

- 11.5. Да се дефинира функция **negate(p)**, където $p : A \rightarrow bool$ е едноместен предикат. **negate** да връща предиката $\neg p$.
- 11.6. Да се дефинира функция **repeated(k,f)**, където $k \geq 0$ е естествено число, а $f : A \rightarrow A$ е едноместна функция. Ако $h = repeated(k, f)$, то $h : A \rightarrow A$ е такава, че $h = f^k(x) = \underbrace{f(f \dots (x))}_k$.

- 11.7. Да се дефинира функцията `createfn(args, values)`, където `args` е вектор с елементи от тип `U`, а `values` е вектор с елементи от тип `V`. Двата вектора са с еднакъв брой елементи. Ако $h = \text{createfn}(\text{args}, \text{values})$, то $h : U \rightarrow V$. По дефиниция $h(u) = v$ т.с.т.к. u е елемент на `args` с индекс i , а $v = \text{values}[i]$ (при повече от едно срещания на u приемаме най-малкия индекс). Ако u не е елемент на `values`, функцията h е недефинирана.
- 11.8. Да се дефинира функцията `switch(n, f, g)`, където $n \geq 1$ е естествено число, а $f, g : A \rightarrow B$ са едноместни функции. `switch` да връща функция $h : A \rightarrow B$, която при първите си n извиквания да дава същите стойности като f , а след това - като g .
- 11.9. Да се дефинира функцията `before(n, f)`, където $n \geq 1$ е естествено число, а $f : A \rightarrow B$ е едноместна функция от произволен тип. `before` да връща функция $h : A \rightarrow B$, която при първите си n извиквания да дава същите стойности като f , а след това - последната върната от f стойност.

Методът на трапеците[8] (Trapezoidal rule) е числен метод за приближено изчисление на стойността на определения интеграл $\int_a^b f(x)dx$ на функцията $f(x)$ в интервала $[a, b]$. Същността на метода е разбиване на интервала $[a, b]$ на определен брой подинтервали $[x_i, x_{i+1}]$. За всеки подинтервал се построява правоъгълен трапец с основа $x_i x_{i+1}$ и бедра, свързващи основата с графиката на функцията в точките $f(x_i)$ и $f(x_{i+1})$ (вж. фиг. 11.2). Лицата на така построените трапеци се сумират, което дава приближение на стойността на определения интеграл. Можете да прочетете повече за метода например тук[8].



Фигура 2.1: Chained trapezoidal rule. Източник: Wikipedia[8]

- 11.10. Да се дефинира функцията `integrate(f, N)`, където f е функция от тип $f : \text{double} \rightarrow \text{double}$, а N е цяло положително число. `integrate` да конструира и връща функция $i(a, b)$ от тип $i : \text{double} \times \text{double} \rightarrow \text{double}$,

изчисляваща приближено стойността на определения интеграл $\int_a^b f(x)dx$ (приемайки, че такъв съществува) по метода на трапеците, по-специално Chained trapezoidal rule[8] с N на брой подинтервала.

12 Линејни едносвързани списъци

Следващите задачи са върху примерния шаблон `LList`, разработен на лекции. Шаблонът реализира прост контейнер чрез линеен едносвързан списък:

```
template <typename T>
struct box
{
    box(const T, box<T>*);
    box();
    T data;
    box<T>* next;
};
template <typename T>
class LList
{
public:
    LList ();
    LList (const LList<T>&);
    LList<T>& operator = (const LList<T>&);
    void push (const T&);
    void pop ();
    void insertAt (size_t, const T&);
    void deleteAt (size_t);
    void print () const;
    ~LList ();
private:
    box<T> *first;
};
```

Следните задачи да се решат като упражнение за директно боравене с указателите и двойните кутии, вместо да се свеждат до използването на вече готови методи от реализацията на класа. Т.е. решенията на задачите да не ползват други методи, освен ако не са помощни функции, специално написани за тях.

- 12.1. Да се реализира метод `int LList<T>::count(int x)`, който преброява колко пъти елементът x се среща в списъка.
- 12.2. Да се реализира конструктор с два аргумента x и y от тип `int`. Конструкторът създава списък с елементи $x, x + 1, \dots, y$, при положение, че $x \leq y$.

- 12.3. Да се реализира метод `LList<T>::push_back` за добавяне на елемент от тип `T` към *края* на списъка.
- 12.4. Да се реализира метод оператор `LList<T>::operator +=` за добавяне на елемент от тип `T` към *края* на списъка.
- 12.5. Да се реализира метод `LList<T>::get_ith(int n)` за намиране на *n*-тия поред елемент на списъка.
- 12.6. Да се реализира метод `LList::removeAll (x)`, който изтрива всички срещания на елемента `x` от списъка.
- 12.7. Да се реализира метод `l1.append(l2)`, която добавя към края на списъка `l1` всички елементи на списъка `l2`.
- 12.8. Да се реализира метод `LList<T>::concat`, който съединява два списъка в нов, трети списък. Т.е. `l1.concat(l2)` създава и връща нов списък от елементите на `l1`, следвани от елементите на `l2`.
- 12.9. Да се дефинират оператори `LList<T>::operator+=` и `LList<T>::operator+`, съответни на методите `append` и `concat`.
- 12.10. Да се дефинира оператор за индексване, позволяващ четене и писане на елемент на даден индекс в списъка.
- 12.11. Да се дефинира метод `LList::reverse`, който обръща реда на елементите на списъка. Например, списъкът с елементи 1, 2, 3 ще се преобразува до списъка с елементи 3, 2, 1.
- 12.12. Да се дефинира функция `map` за списъци във функционален и деструктивен вариант.
- 12.13. Да се дефинира функция `reduce` за списъци.
- 12.14. За шаблона на клас `LList` да се разработят конструктор за копиране, оператор за присвояване и деструктор.

13 Клас символен низ

Следващите задачи са върху примерния клас, реализиращ символен низ, разработен на лекции. Шаблонът реализира символен низ в динамичната памет:

```
class String
{
public:
    String ();
    String (const String<T>&);
    String (const char*);
    String& operator = (const String&);
    ~String();

    char operator [] (size_t) const;
    bool operator != (const String&) const;

    friend std::ostream &operator<<(std::ostream &, const String &);
    ~String ();
private:
    char *str;
};
```

- 13.1. Да се реализира метод `substring (startIndex, endIndex)` на клас `String`, намиращ подниз с дадено начало и край.
- 13.2. Да се реализира метод `substring (s)` на клас `String`, който намира индекса на първото срещане на подниз `s` в дадения низ, или `-1`, ако `s` не е подниз на дадения низ.
- 13.3. Да се реализира метод `split(char separator)` на клас `String`. Методът да връща `std::vector` от символни низове (обекти от клас `String`), които се получават като се раздели на части дадения низ според дадения разделител. Например, низът *"Hello wold, have a nice day!"* с разделител символа `,` ще генерира масива `{"Hello world", " have a nice day!"}`.
- 13.4. Да се добавят оператори `+=` към класа `String`, позволяващи добавяне към края на низа на: символ (`char`), цяло число (`int`), булева стойност (`bool`).
- 13.5. Да се добави оператор `=` към класа `String`, позволяващ на низ да се присвои произволен вектор с елементи от произволен тип `T`. Резултатният низ `s` от присвояването `s = v` да има формата `"{v1, ..., vk}"`, където

$v_1, ..v_k$ са представянията на елементите на v като низове. Приемаме, че типа T е някой от типовете, поддържани от оператора $+=$ на клас `String`.

- 13.6. (*) Да се реализира оператор за вход от поток. *Внимание: Да се съобрази, че броят на въведените символи от оператора `>>` за `char[]` не е предварително известен!*

14 Наследяване и полиморфизъм

14.1 Наследяване, виртуални методи и абстрактни класове

14.1. Шахматни фигури.

- а) Да се дефинира структура `ChessPosition`, описваща коректна позиция на фигура върху шахматна дъска ('A'-'H',1-8). Да се дефинира абстрактен клас `ChessPiece`, описващ шахматна фигура със следните операции:
- `ChessPosition getPosition ()`: Дава позицията на фигурата на дъската
 - `[подходящ тип] allowedMoves ()`: Дава списък с всички възможни позиции, до които дадена фигура може да достигне с един ход
 - `bool wins (ChessPosition)`: Проверява дали фигурата “владее” дадена позиция, т.е. дали позицията е в списъка с възможни ходове на фигурата
- б) Да се дефинират класовете `Rook` и `Knight`, наследници на `ChessPiece`, описващи съответно шахматните фигури топ и кон.
- в) “Стабилна конфигурация” наричаме такава подредба на фигурите по дъската, при която никоя фигура да не е върху позволен ход на друга фигура (т.е. никои две фигури не се “бият”). Да се дефинира функция `allMoves ([подходящ тип] pieces[, ...])`, която по списъка `pieces`, съдържащ произволен брой разнородни шахматни фигури, отпечатва на конзолата всеки възможен ход на фигура от `pieces` такъв, че след изпълнението му списъка с фигури представлява стабилна конфигурация. Информацията за ходовете да съдържа типа на фигурата, старата позиция и новата позиция, например:

`Queen A1 -> B2`

`Knight B3 -> A5`

Забележка: Реализирайте всички конструктори и други операции, които смятате, че са необходими на съответните класове.

Забележка: Под “списък” се има предвид обект от класовете за линеен едносвързан списък или динамичен масив, разработени на лекции, или който е да е друг тип, който познавате и който представлява контейнер за обекти.

14.2. Походова игра.

Нека `GameBoard` е дадена квадратна матрица $N \times N$ от цели числа, всеки елемент на която има стойност 0, 1 или 2. Елементите на матрицата със стойност 0 наричаме “земя”, тези със стойност 1 наричаме “огън”, а тези със стойност 2 - “вода”. `GameBoard` ще наричаме “игрова дъска”. Под “съседна позиция” на позицията (i, j) ще разбираме тези елементи на матрицата (i', j') , такива че $|i - i'| \leq 1$ и $|j - j'| \leq 1$.

В условието по-долу N и `GameBoard` да се приемат за предварително дефинирани глобални променливи.

- а) Да се дефинира структура `Position`, описваща позиция на игровата дъска, задаваща реда и колоната на нейн елемент. Да се дефинира абстрактен клас `GamePlayer`, който описва играч на игровата дъска със следните операции:
- `Position getPosition ()`: Дава позицията на играча на дъската
 - `[подходящ тип] allowedMoves ()`: Дава списък с всички възможни позиции, до които даден играч може да достигне с един ход
 - `bool wins (Position)`: Проверява дали играча “владее” дадена позиция, т.е. дали позицията е в списъка с възможни ходове на играча
- б) Да се дефинира клас `Knight`, наследник на `GamePlayer`, описващ “сухопътен рицар”. Позволените ходове на сухопътния рицар се задават със следното правило: Рицарят може да се премести в позициите, които са съседни на текущата му и които:
- са земя
 - нямат огън в съседство
- в) Да се дефинира клас `SeaMonster`, наследник на `GamePlayer`, описващ “морско чудовище”. Позволените ходове на морското чудовище се задават със следното правило: Морското чудовище може да се премести във всички позиции, които са достижими от неговата при придвижване по хоризонтала или вертикала, което преминава само през вода. Например, ако вдясно от играча има три поредни позиции с вода, следвани от една позиция земя, то и трите водни позиции са достижими, но земната и всички вдясно от нея - не.
- г) Всяко морско чудовище да има “сила”: цяло число k , което показва колко е максималния брой позиции по вертикала и хоризонтала, които чудовището може да преплува на един ход. Силата да се задава при конструиране на чудовища.
- д) “Стабилна конфигурация” наричаме такава подредба на играчите по дъската, при която никой играч да не е върху позволен ход

на друг играч (т.е. никои два играча не се “бият”). Да се дефинира функция `allMoves` ([подходящ тип] `players[, ...]`), която по списъка `players`, съдържащ произволен брой разнородни играчи, отпечатва на конзолата всеки възможен ход на играч от `players` такъв, че след изпълнението му списъка с играчи представлява стабилна конфигурация. Информацията за ходовете да съдържа типа на играча, старата позиция и новата позиция, например:

`Knight (0,0) -> (1,1)`

`SeaMonster (2,2) -> (5,2)`

Забележка: Реализирайте всички конструктори и други операции, които смятате, че са необходими на съответните класове.

Забележка: Под “списък” се има предвид обект от класовете за линеен едносвързан списък или динамичен масив, разработени на лекции, или който е да е друг тип, който познавате и който представлява контейнер за обекти.

- 14.3. Задача 2.4.25[7] В софтуерна фирма има два вида служители – програмисти и мениджъри. Отдел “личен състав” поддържа следната информация за всеки от програмистите:

- име;
- стаж (в месеци);
- дали знае C++;
- дали знае C#

и следната информация за всеки от мениджърите:

- име;
- стаж (в месеци);
- колко човека управлява.

Да се напише програма, която позволява на отдел “личен състав” да поддържа списък с всички програмисти и мениджъри във фирмата. Програмата да може да извършва следните операции:

- постъпване на нов служител;
- напускане на служител;
- извеждане на списък с данни за всички служители.

Да се въведат примерни данни за служители в софтуерна фирма и над тях да се изпълнят следните операции:

- изтриване на всички служители, които имат стаж по-малко от 3 месеца;
- записване на данните в текстов файл;
- прочитане на данните от текстов файл;

14.2 Сериализация и десериализация на контейнери. Шаблон “Фабрика”

- 14.4. Да се сериализира и десериализира масив от масиви от числа:

```
vector<vector<int> >
```

Да дефинират необходимите за целта оператори. Да се тества програмата!

- 14.5. Да се сериализира и десериализира масив от масиви от символни низове (`char*`):

```
vector<vector<char*> >
```

Да дефинират необходимите за целта оператори. Да се тества програмата!

- 14.6. Да се разработи клас `ChessGame`, описващ шахматна игра с ножество от фигури по дефиницията на задача 14.1. и имената на двамата играчи. Функциите за игра от задачата да се преобразуват до методи на `ChessGame`. Да се сериализират и десериализират шахматни игри.

- 14.7. Да се разработи клас `BoardGame`, описващ походова игра с ножество от фигури по дефиницията на задача 14.2. и имената на двамата играчи. Функциите за игра от задачата да се преобразуват до методи на `BoardGame`. Да се сериализират и десериализират походови игри.

- 14.8. Да се реализира абстрактен клас `NetworkDevice`, който дефинира следните (абстрактни, чисто виртуални) операции:

- `bool attachTo(NetworkDevice* device)`: свързва устройството с друго устройство `device`.

Реализациите на метода в наследниците на `NetworkDevice` ще връщат `true`, ако свързването е възможно и `false`, ако свързването не е възможно. Правилата, по които се определя дали може или не може да се свърже устройството, зависят от конкретния наследник на `NetworkDevice`.

При дефиниране на метода в наследените класове осигурете, че създадената връзка е двупосочна. Т.е. счита се, че ако устройството А е свързано с устройството В, то и устройството В е свързано с устройството А. Не допускайте дадено устройство да може да се свърже повече от веднъж с едно и също устройство или пък да се свърже със себе си.

- [попълнете правилния тип] `getAttachedDevice(int i)`. Връща (*Упътване: указател към*) *i*-тото поред устройство, към което даденото устройство е свързано и `nullptr`, ако индексът *i* не е валиден.

Класът `NetworkDevice` също да съдържа и уникален идентификатор от тип `int` за всяко устройство. Идентификаторът може да се задава чрез конструкторите на наследниците.

Да се реализират производните класове `EndDevice` и `Switch`.

- За `EndDevice` е характерно, че устройството може да е свързано най-много с още едно устройство. Т.е. във всеки момент от времето `EndDevice` или не е свързан с нищо, или е свързан с точно едно устройство. Веднъж свързано, `EndDevice` не може да бъде свързано отново с друго устройство.
- За `Switch` е характерно, че устройството може да е свързано с максимум 8 други устройства. При достигане на броя на свързаните устройства до 8, устройството `Switch` не може да бъде свързано с повече устройства.

Промяна на вече създадена връзка не е възможна и при двата вида устройства.

За така дефинираните класове да се решат следните задачи:

- а) Да се реализира функция `void printConnections(NetworkDevice* devices[], int n)`, която отпечатва на екрана информация за връзките на всяко устройство в масив от устройства. *Упътване: добавете нова виртуална функция `printConnnection` в базовия клас. Връзките можете да печатате във формата <идентификатор 1> -- <идентификатор 2>.*
- б) Да се създаде примерна програма, в която се инициализират няколко устройства, добавят се в масив, създават се връзки между тях и се използва функцията `printConnections` за отпечатване на връзките.
- в) Да се реализира функция `bool connected([попълнете правилния тип] d1, [попълнете правилния тип] d2)`, която проверява дали има връзка (пряка или косвена) между устройствата `d1` и `d2`. За улеснение приемере, че няма циклични връзки между устройствата. *Упътване: използвайте рекурсивна функция по подобие на функциите за търсене на път.*
- г) *Допълнителна задача:* Решете горната задача и при случая на възможни циклични връзки.
- д) *Допълнителна задача:* Напишете функция, която по масив от устройства търси дали има циклична връзка между някои две от тях.
- е) *Допълнителна задача с повишена трудност:* Сериализирайте и десериализирайте масив от свързани устройства.

14.3 Виртуални конструктори и деструктори

- 14.9. Да се реализира абстрактен базов клас **Set**, описващ множество от цели числа (**int**) с една единствена операция:

```
bool member(int x)
```

Операцията проверява дали дадено число е елемент на множеството. Да се реализират следните наследници на **Set**:

- а) **EvenNumbers**: множеството на всички четни числа от тип **int**
- б) **DynamicSet**: множество, което позволява добавяне на елементи чрез операция **addElement(int)**
- в) **SetUnion**: обединение на произволен брой множества с операция **addSet([подходящ тип] s)**, където **s** може да е от всеки един от трите вида множества(*)

Да се реализират необходимите конструктори и деструктори, включително и виртуални такива.

(*)Множеството **s** може да се добави към обединението във вида, в който е при извършване на **addSet**. Т.е. бъдещи промени по **s** не е нужно да имат ефект върху обединение, което съдържа **s**.

- 14.10. За класа **ChessGame** от задача 14.6. да се разработят подходящи конструктори, деструктори и оператори за присвояване.
- 14.11. За класа **BoardGame** от задача 14.7. да се разработят подходящи конструктори, деструктори и оператори за присвояване.
- 14.12. Да се реализира копиране и изтриване на мрежови устройства (клас **NetworkDevice**) от зад 14.8.. Приемаме, че в мрежата няма цикли.

14.4 Наследяване и агрегация. Йерархични дървета от обекти, съдържащи други обекти. Шаблон за дизайн “Посетител.”

Проблемите около агрегацията на обекти от същата йерархия и получаващите се йерархични дървета от обекти, съдържащи други обекти, са демонстрирани чрез йерархия от геометрични фигури, наследници на абстрактен базов клас `Shape`.

```
class Shape
{
public:
    Shape ();
    Shape (int x,int y,const char *title);
    Shape (const Shape&);
    virtual Shape* clone () = 0;
    void set_x (int);
    void set_y (int);
    void set_text (const char*);
    void operator = (const Shape& s);
    int get_x();
    int get_y();
    char* get_text();
    virtual ~Shape();
protected:
    int x;
    int y;
    char *text;
};
```

Към йерархията е добавен клас `Group`, който освен че е наследник на `Shape`, съдържа и контейнер от наследници на `Shape`:

```
class Group: public Shape
{
public:
    //...
    void addShape (Shape*);
    size_t get_nChildren () const;
    Shape* getChild (size_t i) const;
    //...
```

```
private:
    std::vector<Shape*> children;
};
```

За изобразяване на фигурите и сериализацията им във файл се ползва шаблонът посетител (Visitor Pattern). Всеки конкретен посетител е наследник на клас:

```
class Visitor
{
public:
    virtual void visitRectangle (Rect*) = 0;
    virtual void visitCircle (Circle*) = 0;
    virtual void visitGroup (Group*) = 0;
};
```

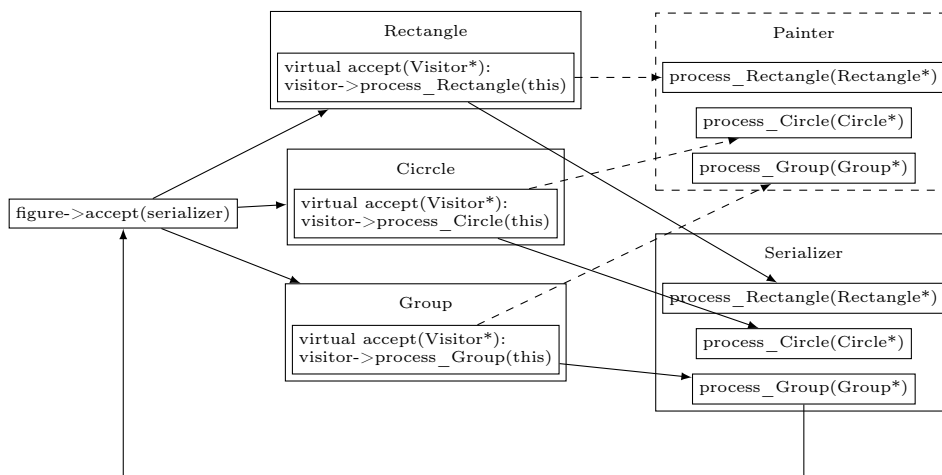
Йерархията от фигури реализира виртуалния метод `void accept (Visitor*)`, като всяка конкретна фигура изпълнява съответния на нея метод на посетителя. По този начин се реализира т.нар. “double dispatch”. Т.е. в следната ситуация:

```
figure->accept(visitor),
```

кой точно метод ще се изпълни се определя динамично не само според типа на обекта, сочен от **figure**, но и според типа на конкретния посетител, сочен от **visitor**. Това е изобразено на фигура 2.2.

- 14.13. Да се реализира сериализация на йерархия от фигури чрез шаблона “Посетител”.
- 14.14. При сериализирането на фигури чрез посетител, представянето на всяка фигура да започва на нов ред. Фигурите, вложени в групи, да са подравнени навътре в реда в зависимост от дълбочината на вложането. Например:

```
circle 0 0 10
group 100 100 3
    circle 0 0 1
    rect 10 10 20 30
    group 0 0 1
        circle 1 1 5
    rect 0 0 10 10
```

Фигура 2.2: Double Dispatch при йерархии от фигури и посетители

- 14.15. Да се резлизира посетител за фигури **Splitter**, чрез който могат да се достъпят отделно вектори от всички кръгове и всички правоъгълници в йерархията, например чрез `split.getCircles()`, `split.getRects()`.

Splitter да съдържа копия на оригиналните фигури. Да се разработят подходящи конструктори, деструктори и оператори за присвояване за **Splitter**.

- 14.16. Да се дефинира посетител **SurfaceCalculator**, който обхожда йерархия от фигури и изчисля сумата на лицата на всички фигури в нея. Приемем, че лицето на група от фигури е равно на сумата на лицата на фигурите в групата.

- 14.17. Да се модифицира посетителя **Painter** от лекции така, че групите да имат цвят. Цветът на групата служи за фон, върху който се изобразяват елементите на групата. Фонът да се изрисува в правоъгълник с горен-ляв връх – координатите на група и страни, успоредни на осите. Размерът на правоъгълника да е минмалният възможен такъв, че всички вложени фигури се съдържат в правоъгълника.

15 Рекурсивен парсер

По време на лекции разработваме лексически анализатор, рекурсивен парсер и интерпретатор на прост език, състоящ се от аритметични изрази от следния вид:

```
<expression> ::= <number> | (<expression> <operator> <expression>)
<number> ::= {0,..,9}+
<operator> ::= + | - | * | /
```

За двата вида възможни изрази – числова константа и приложение на двуместен аритметичен оператор, оградено в скоби – дефинираме съответно класовете `ExprConstant` и `ExprOperator`, представящи изразите в синтактично дърво. Двата класа са наследници на абстрактен базов клас `Expression`, дефиниращ основаната операция `value` за намиране на оценка на израз.

Езикът разширяваме с условен оператор от вида:

```
<expression> ::= ... | if <expression> then <expression> else <expression>
```

Изразите от този вид се представят в синтактичното дърво с обекти от клас `ExprIf`, съдържащ три член данни от тип `Expression*`, `condition`, `expthen` и `expelse`. Методът `value` връща оценката на `expthen` или `expelse` в зависимост от оценката на `condition`.

- 15.18. Интерпретаторът на аритметични изрази от лекции да се разшири с възможност за групиране на изрази в “блокове”, започващи с ключовата дума `begin` и завършващи с ключовата дума `end`:

```
<expression> ::= ... | begin (<expression>)+ end
```

Оценката на блок от изрази да се състои в последователно оценяване на всички изрази в блока и използване на оценката на последния израз от блока като оценка на целия блок. Например: `begin 1; 2; 3; end` има оценка 3.

- 15.19. Интерпретаторът на аритметични изрази от лекции да се разшири с възможност за извеждане на стойност на израз на стандартния изход:

```
<expression> ::= ... | print <expression>
```

Оценката на израз от типа `print` е се намира като оценката на израза е, като освен това се произвежда извеждане на стандартния изход като страничен ефект. Всяка отделна стойност да бъде изведена на нов ред.

Примерер: Оценката на израза `print 1` ще е 1 и ще доведе до извеждане на “1” на стандартния изход. Оценката на

```
begin
  print 1;
  print 2;
  print begin 3; 4; end;
end
```

ще доведе до извеждане на следните редове на стандартния изход:

```
1
2
4
```

- 15.20. Нека е дадена следната граматика на език за програмиране, зададена чрез BNF:

```
<program> ::= (<statement>;)+
<statement> ::= <assignment> | <output>
<assignment> ::= set <variable> = <expression>
<output> ::= print <expression>
<variable> ::= a..z
<expression> ::= <variable> | <number> | <complex-exp>
<complex-exp> ::= <expression>(<operator> <expression>)*
<complex-exp> ::= (<expression>(<operator> <expression>)+)
<operator> ::= + | - | * | /
```

Примерни програми на този език и съответния резултат на стандартния изход са:

```
print 10;
```

Изход: 10

```
set x = 3*(1+2);
print x + 10;
```

Изход: 20

```
set x = 10;
set y = 5*5;
print x*y*2;
```

Изход: 500

Да се напише интерпретатор за дадения език.

- 15.21. Към интерпретатора от предишната задача да се добави възможност за стандартен вход, условен оператор и аритметични сравнения:

```

<program> ::= (<statement>;)+
<statement> ::= <assignment> | <output> | <input> | <condition>
<input> ::= read <variable>
<condition> ::= if <expression> then <statement>
<condition> ::= if <expression> then <statement> else <statement>
<operator> ::= + | - | * | / | < | > | =

```

Пример: следната програма намира броя на корените на квадратно уравнение:

```

read a;
read b;
read c;
set D = b * b - 4 * a * c;
if D > 0 then print 2;
if D = 0 then print 1 else print 0;

```

16 Обработка на изключения

16.1. Да се добавят подходящ изключенията за методите за добавяне и изтриване на елементи на линейния едносвързан списък от секция 12.

16.2. При десериализацията на класа `BoardGame` от задача 14.7. да се хвърлят подходящи изключения, сигнализиращи за

- Непознат тип играч
- Грешни данни за даден играч (напр. липсва сила за мосрко чудовище)
- Невъзможна конфигурация (напр. два играча на еднаква позиция)
- Други

Ако е подходящо, да се реализират наследници на `std::exception`.

Програмата да съобщава грешките на потребителя.

(*)Какво трябва да са направи, за да се посочи реда от файла, на който е открита грешката?

16.3. Решението на задача 16.2. да се разшири така, че да позволява възстановяване от грешки. При грешка при десериализацията на някой играч, да се изведе съобщение на потребителя и да се направи опит да се продължи с прочитането на останала част от дефиницията на играта.

Глава 3

Структури от данни

16 Линеен едносвързан и двусвързан списък

16.1 Представяне на двусвързан списък

Възел на линеен двусвързан списък представяме със следния шаблон на структура:

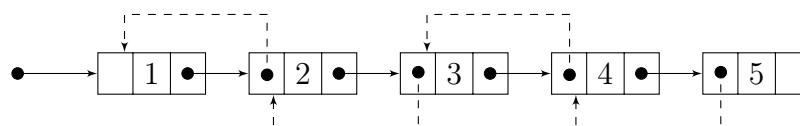
```
template <class T>
struct dllnode
{
    T data;
    dllnode<T> *next, *previous;
};
```

Освен ако не е указано друго, задачите по-долу да се решат като се реализират методи на клас `DLList` със следния скелет:

```
template <class T>
class DLList
{
    //...
private:
    dllnode<T> *first, *last;
};
```

Преди да пристъпите към задачите, реализирайте подходящи конструктори, деструктор и оператор за присвояване на класа.

Следните задачи да се решат като упражнение за директно боравене с възлите на линеен двусвързан списък. Функциите (методите) да се тестват с



Фигура 3.1: Двусвързан списък

подходящи тестове.

- 16.1. Да се дефинира функция `int count(dllnode<T>* l, int x)`, която преброява колко пъти елементът x се среща в списъка с първи елемент l .
- 16.2. Функция `dllnode<int>* range (int x, int y)` която създава и връща първия елемент на списък с елементи $x, x + 1, \dots, y$, при положение, че $x \leq y$.
- 16.3. Да се дефинира функция `removeAll (dllnode<T>*& l, const T& x)`, която изтрива всички срещания на елемента x от списъка l .
- 16.4. Да се дефинира функция `void append(dllnode*<T>& l1, dllnode<T>* l2)`, която добавя към края на списъка l_1 всички елементи на списъка l_2 . Да се реализира съответен оператор `+=` в класа на списъка.
- 16.5. Да се дефинира функция `dllnode* concat(dllnode<T>* l1, dllnode<T>* l2)`, който съединява два списъка в нов, трети списък. Т.е. `concat(l1, l2)` създава и връща нов списък от елементите на l_1 , следвани от елементите на l_2 . Да се реализира съответен оператор `+` в класа на списъка.
- 16.6. Да се дефинира функция `reverse`, която обръща реда на елементите на списък. Например, списъкът с елементи 1, 2, 3 ще се преобразува до списък с елементи 3, 2, 1.
- 16.7. Да се напише функция `void removeduplicates (dllnode *&l)`, която изтрива всички дублиращи се елементи от списъка l .
- 16.8. За шаблона `DLList` да се реализира изтриване на елемент по индекс.
- 16.9. Да се реализира метод `mergeWithSorted(const DLList<T>&)` на шаблона `DLList`.

a.mergeWithSorted(b) да слива сортирано списъците **a** и **b**, като резултатът от сливането се получава в списъка **a**. Приема се, че елементите и на двата списъка са в нарастващ ред спрямо оператора $\leq$ на типа T , като елементите на резултата също трябва да са в нарастващ ред. Сливането да се извършва за линейно време.

Пример: при сливането на списъците $\{1, 3, 5, 7\}$ и $\{2, 4, 6\}$ се получава списъка $\{1, 2, 3, 4, 5, 6, 7\}$.

16.2 Списъци и сложности

Функцията `std::clock()` от `<ctime>` връща в абстрактни единици времето, което е изминало от началото на изпълнение на програмата. Обикновено тази единица за време, наречена “tick”, е фиксиран интервал “реално” време, който зависи от хардуера на системата и конфигурацията ѝ. Константата `CLOCKS_PER_SEC` дава броя tick-ове, които се съдържат в една секунда реално време.

Чрез следния примерен код може да се измери в милисекунди времето за изпълнение на програмния блок, обозначен с “...”.

```
clock_t start = std::clock();
//...
clock_t end = std::clock();

long milliseconds = (double)(end-start)/
                    (CLOCKS_PER_SEC/1000.0);
```

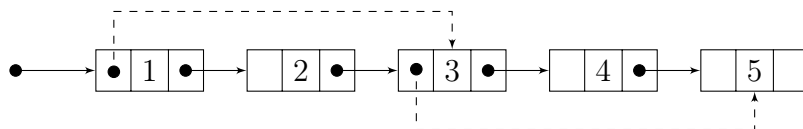
- 16.10. За шаблона `DLList` да се дефинира метод `size_t find(const T& x)`, който намира поредния номер на елемента `x` в списъка (или връща размера на списъка, ако такъв елемент няма). Да се напише подходящ тест и да се изследва времевата сложност на метода емпирично.
- 16.11. Да се изпробват поне две различни стратегии за разширяване на динамичен масив (например, увеличаване на размера с 1 и с коефициент). Да напишат подходящи тестове и да се сравнят производителностите на двата подхода емпирично.

16.3 SList (опростен вариант на Skip List)

Разглеждаме *опростена* реализация на структурата от данни Skip List (“Списък с прескачане, СП”). Възелът на линейния едносвързан списък разширяваме с още един указател към следващ елемент:

```
template <class T>
struct lnode
{
    T data;
    lnode<T> *next, *skip;
};
```

Както и при стандартния едносвързан списък, всеки от елементите на СП



Фигура 3.2: Списък с прескачане

съдържа в указателя **next** адреса на непосредствения си съсед. Елементите на списъка са под дадена наредба. Някои от елементите могат да съдържат в указателя **skip** адреса на друг елемент, намиращ се по-напред в редицата от елементи (вж. Фигура 3.2). Например, нека имаме СП с n елемента в нарастващ ред. Ако списъкът е построен така, че всеки $\sqrt{n}$ -ти елемент има указател към следващия $\sqrt{n}$ -ти елемент, то търсенето на подписъка, съдържащ даден елемент, ще бъде със сложност $O(\sqrt{n})$. Търсенето в рамките на подписъка с $\sqrt{n}$ елемента остава линейно. Идеята може да се продължи така, че всеки елемент да може да има и по-голям брой указатели към елементи все по-напред в СП, но за нашите цели ще се ограничим до описания прост СП.

Следващите задачи изискват реализация на клас **SList** с основните му канонични методи и метод за построяване на “бързите връзки”. Реализирайте обикновен метод за вмъкване на елементи **insertSorted**, който вмъква елементи грижейки се само за непосредствените връзки (**next**), и метод **speedup**, който построява бързите връзки в списъка, след като в него са вмъкнати определен брой елементи.

- 16.12. Да се реализират конструктор за копиране и оператор на присвояване на **SList**, които репликират и бързите връзки.
- 16.13. Да се подобри методът за вмъкване на елемент **insertSorted**. Ако при вмъкване на нов елемент някой подписък (обособен от дадена бърза връзка) получи повече от $2\sqrt{n}$ елемента, да се добави нова бърза връзка така, че подписъка да се раздели на два подписъка.
- 16.14. Да се извърши времево измерване на проблема за търсене на елемент в подреден **SList**, както е обяснено в Секция 16.2, и да се изобрази чрез графика. Да се извършат емпирични сравнения на производителността на търсенето със и без оптимизацията.

17 Итератори, интервали и адаптери

17.1 Итератори за линейни СД

Следните задачи да се решат като упражнение за работа с итератори. Някои от задачите изискват реализация на клас динамичен масив, линеен едносвързан списък, линеен двусвързан списък и `forward` итератори за тях. Всяка функция да се тества с подходящи тестове върху поне два вида контейнери. Има ли разлика в производителността за някои от тях в зависимост от избора на контейнер?

- 17.1. Да се разработи шаблон на клас `Range<T>`, позволяващ итерация над елементите на интервал от последователни стойности с помощта на `forward iterator`. Например:

```
Range<int> r{0,10};
for(int i : r) {std::cout << i << std::endl;};
```

- а) За типа `T` да се приеме, че допуска операцията събиране с единица `x+1`
- б) Класът да допуска инициализация с функция, която намира следващ елемент от интервала. Следният пример ще изведе четните числа в интервала `[0..10]`:

```
Range<int> r{0,10,[](int x)->int{return x+2;}};
for(int i : r) {std::cout << i << std::endl;};
```

- 17.2. Да се разработи шаблон на клас `ArrayRange<T>`, с чиято помощ могат да се итерират първите N елемента на масив `T[]` по следния начин:

```
int example[] = {1,2,3,4,5,6,7,8,9,10};
for(int i : ArrayRange<int>(example,5)) {std::cout << i << std::endl;};
```

- 17.3. Да се разработи шаблон на клас `Repeat<T>`, с чиято помощ по дадени елемент $a \in T$, функция $f : T \rightarrow T$ и предикат $p : T \rightarrow bool$, може да се обходи редицата $a, f(a), f(f(a)), \dots, f^k(a)$, такава, че за всеки елемент на редицата е изпълнен предиката p , но не е вярно, че $p(f^{k+1}(a))$. Т.е. най-дългата редица от последователни приложения на функцията f над аргумент a до нарушаване на някакво условие. Следният пример ще изведе четните числа в интервала `[0..100]`:

```
Repeat<int> r{0,
              [](int x)->int{return x+2;},
              [](int x)->bool{return x <= 100;}};
for(int i : r) {std::cout << i << std::endl;};
```

- 17.4. Да се разшири интераторът на динамичен масив така, че да поддържа оператора за стъпка назад `--`.
- 17.5. Възможно ли е да се разшири интераторът на линеен едносвързан списък така, че да поддържа оператора за стъпка назад `--`? Ако да, какви са слабостите на реализацията и как могат да се решат?
- 17.6. Да се разшири интераторът на линеен двусвързан списък така, че да поддържа оператора за стъпка назад `--`.
- 17.7. Да се реализира итератор за клас символен низ от секция 13 така, че да може със следният цикъл:

```
for (char c : s) {...}
```

да се “обхождат” символите на `s`, където `s` е символен низ.

- 17.8. Да се дефинира функция `map`, която прилага едноаргументна функция $f : int \rightarrow int$ към всеки от елементите на произволен контейнер с числа (`int`). Да се дефинира и шаблон на функцията за контейнер с произволен тип на елементите.
- 17.9. Да се напише функция `bool duplicates (...)`, която проверява дали в контейнер има дублиращи се елементи.
- 17.10. Да се напише функцията `bool issorted (...)`, която проверява дали елементите на даден контейнер са подредени в нарастващ или в намаляващ ред.
- 17.11. Да се напише функцията `bool palindrom (...)`, която проверява дали редицата от елементите на даден контейнер обръзва палиндром (т.е. дали се четат еднакво както отляво надясно така и отдясно наляво).

17.2 Интервали и адаптери

Нека “интервал” да наричаме последователни елементи на някакъв контейнер, зададени чрез итератори към началото и края на последователността. С помощта на следния шаблон:

```
template <class Iterator>
class Range{
public:

    Iterator begin()
    {return _begin;}
    Iterator end()
```

```

    {return _end;}

    Iterator _begin;
    Iterator _end;
};

```

става възможно не само да групираме двойката начало-край в общ обект, но и да итерираме съответната поредица с цикъл `for`, както бихме итерирали елементите на контейнер: Ако `begin` и `end` са съответните итератори към елементи от тип `E` на някакъв контейнер (или друг интервал) и имаме обекта `Range<SomeIterator> range(begin,end)`, то можем да направим обхождането:

```
for (E& x : range){...}
```

Разглеждаме трансформации на интервали, които подменят техните итератори с варианти, променящи начина, по който се достъпват елементите на интервала. Можем да построим шаблон `MappedIterator<Iterator>` (`it`), който обвива обект `it` от даден тип `Iterator` и делегира операциите `++` и `!=` към `it`, но подменя операцията `*` с такава, която връща резултат от приложение на някаква функция `f` над резултата от `*it`, т.е. връща `f(*it)`. Можем да построим и `FilteredIterator<Iterator>`, който пък изменя реда, в който се достъпват елементите чрез някакъв итератор така, че `*` да връща само елементите, удовлетворяващи даден предикат.

```

template <class Iterator>
class MappedIterator
{
public:
    using value_type = typename I::value_type;
    std::function<value_type(value_type)> op;
    I original;

    MappedIterator<I>& operator++()
    { ++original; return *this; }
    value_type operator* ()
    { return op(*original); }
    bool operator != (const MappedIterator& it)
    { return original != it.original;}
};

```

И накрая, дефинираме шаблон на “адаптор”, който представлява обект, съдържащ необходимата информация за трансформирането на интервал от някакъв тип чрез подмяна на подлежащите му итератори. За прилагането на `map`, това е едноместната функция, която се прилага над всеки елемент от интервала, но най-вече, типовете на оригиналния и на резултатния интервал. Т.е. смисъла на следния шаблон е да “атоматизира” получаването от интервали от даден тип на интервали от адаптирания тип:

```
template <class Val>
class MapAdaptor
{
    public:
    template <class OriginalIt>
    using adapted_it = MappedIterator<OriginalIt>;
    std::function<Val(Val)> op;

    template <class I>
    Range<MappedIterator<I>> adapt (Range<I> r)
    { return Range<MappedIterator<I>> ({op,r._begin},
                                         {op,r._end}); }
};
```

Аналогично постъпваме и с `filter`.

```
template <class Val>
class FilterAdaptor
{
    public:
    template <class OriginalIt>
    using adapted_it = FilterIterator<OriginalIt>;
    std::function<bool(Val)> pred;

    template <class I>
    Range<FilterIterator<I>> adapt (Range<I> r)
    { return Range<FilterIterator<I>> ({r._begin,r._end,pred},
                                         {r._end,r._end,pred}); }
};
```

Групата шаблони позволява да дефинираме следния оператор:

```
template <class I, class Adaptor>
Range<typename Adaptor::template adapted_it<I>>
    operator | (Range<I> r, Adaptor a)
{
```

```
    return a.adapt(r);
}
```

С негова помощ е възможно да получим вторите степени, увеличени с единица, на четните числа от интервала **range** по следния начин:

```
for (int x : range | filtereven | square | inc){...}
```

където **filtereven**, **square** и **inc** са съответните адаптори:

```
FilterAdaptor<int> filtereven([](int x)->bool{return x%2==0;});
MapAdaptor<int> square ([](int x)->int{return x*x;});
MapAdaptor<int> inc ([](int x)->int{return x+1;});
```

- 17.12. Да се дефинира адаптор **take** с параметър **k**, който трансформира интервал като оставя в него само първите му **k** елемента.
- 17.13. Да се дефинира адаптор **duplicate**, който “удвоява” интервала $a_1, a_2, \dots, a_k$ до интервала $a_1, a_1, a_2, a_2, \dots, a_k, a_k$ (т.е. всеки от елементите се получава по два пъти последователно).
- 17.14. Да се дефинира адаптор **repeat** с параметър **k**, който генерира интервал чрез прилагане чрез **k**-кратно долепяне на оригиналния интервал до себе си, т.е. интервалът $a_1, \dots, a_n$ се превръща в интервала $\underbrace{a_1, \dots, a_n, \dots, a_1, \dots, a_n}_k$.
- 17.15. Да се дефинира адаптор **loop**, който генерира безкраен интервал чрез прилагане на безкрайно долепяне на оригиналния интервал до себе си.
- 17.16. Да се дефинира адаптор **reverse**, който генерира интервал чрез нареждане на елементите на даден интервал в обратен ред (приемаме, че оригиналния итератор притежава операция **--**).
- 17.17. Да се дефинира оператор **a+b** над интервали, който създава нов интервал чрез конкатенацията на интервалите **a** и **b**.
- 17.18. Да се дефинира оператор **a*b** над интервали, който създава нов интервал, състоящ се от тези елементи на **a**, които са елементи и на **b**.
- 17.19. Да се дефинира оператор **a&b** над интервали, където елементите на **a** и **b** са подредени в нарастващ ред. Операторът да създава нов интервал чрез сортирано сливане на елементите на **a** и **b**.

18 Приложения на структурата от данни стек

18.1 Общи задачи за стекове

- 18.1. Нека е даден масив с n елемента. За всеки от елементите на масива да се изведе следващия в масива по-голям елемент. Т.е. за всеки елемент $a[i]$ да се отпечата $a[j]$ такъв, че $n > j > i$ и $\forall k \in (i, j) : a[k] \leq a[i]$. Ако такъв елемент няма, да се изведе числото -1 . Пример, за масива $\{4, 5, 2, 25\}$, да се изведат двойките $(4, 5)$, $(5, 25)$, $(2, 25)$, $(25, -1)$. Алгоритъмът да работи с линейна сложност спрямо n . *Упътване: Задачата е известна под името "Next Greater Element(NGE)".*

18.2 Изрази и стекове

- 18.2. Даден е израз, който може да съдържа отварящи и затварящи скоби. Да се напише функция, която проверява дали скобите на израза са правилно балансирани. Например, изразът $(x+(y+(1+2)))$ считаме за правилно балансиран, но не и израза $(x+y)*3+(x+(1+2))$.
- 18.3. Да се реши горната задача при положение, че изразът може да има едновременно кръгли, фигурни и квадратни скоби.

По време лекции разглеждаме реализация на *Shunting Yard* алгоритъма за преобразуване на инфиксен аритметичен израз в постфиксен вид (Обратен полски запис). Реализацията работи само с двуместни оператори и скоби, като отчита приоритет на операциите.

- 18.4. При изчисляването на стойността на постфиксен израз, да се разгледа случая на ляво-асоциативни оператори (като деление и изваждане).
- 18.5. При изчисляването на стойността на постфиксен израз, да се позволи използването на едноместни функции без влагане. Да се поддържа ограничен набор от предефинирани функции (напр. $\sin(x)$ и $\sqrt{x}$). Функциите да са само едноместни и да не се позволява влагане на извиквания.
- 18.6. Да се преработи алгоритъма за изчисляването на стойността на постфиксен израз така, че да се генерира прав полски запис (префиксна нотация).

18.3 Цикъл със стек вместо рекурсия

Упътване: Решете задачите с рекурсия и след това преобразувайте решението в решение със стек.

- 18.7. (*) Да се дефинира функция за намиране на стойността на полинома на Ермит $H_n(x)$ (x е реална променлива, а n неотрицателна цяла променлива), дефиниран по следния начин:

$$H_0(x) = 1$$

$$H_1(x) = 2x$$

$$H_n(x) = 2xH_{n-1}(x) + 2(n-1)H_{n-2}(x), n > 1,$$

за дадени n и x с използване на стек.

- 18.8. Нека е дадена абстрактна шахматна дъска с размери $n \times n$, $4 \leq n \leq 8$ и число k , $0 \leq k \leq n$. Казваме, че разположени на дъската k коня образуват “валидна конфигурация”, ако никоя фигура не е поставена на поле, което се “бие” от друга фигура според съответните шахматни правила.

Да се дефинира клас `KnightConfig`, представящ “конфигуратор” на шахматни коне. Конструкторът на класа инициализира конфигуратора с числата n и k . Класът позволява “обхождането” една по една на всички валидни конфигурации за дадените параметри, по подобие на `forward` итератор на структура от данни. Класът да притежава следните методи:

- `void KnightConfig::printCurrentConfig()`: Отпечатва текущо намерената конфигурация. Пример за отпечатаната конфигурация с $n = 5, k = 2$:

```

- - - - -
- - Н - -
- - - - -
- - - - Н
- - - - -

```

- `void KnightConfig::findNextConfig()`: Намира следваща конфигурация.
- `bool KnightConfig::noMoreConfigs()`: Показва дали всички възможни конфигурации са вече изчерпани.

- 18.9. Да се реши задачата за Ханойските кули с използване на стек.

Да се дефинира клас `NanoyPlayer` със следните методи:

- Конструктор с параметър, указващ броя дискове върху лявото колче за началното състояние на играта.
- Метод `bool final()`, който показва дали играта е достигнала финално състояние (т.е. всички дискове са на дясното колче).

- Метод `makeMove()`, който извършва един ход от играта.
- Метод `printBoard()`, който отпечатва текущото състояние на игровата дъска, например по следния начин:

```
2
3    1
5 * 4
```

На примера е изобразено състояние на играта, при което на лявото колче има три диска - с размери 5, 3 и 2, на средното колче няма дискове, а на дясното има два диска - с размери 4 и 1.

19 Двоични дървета

19.1 Прости обхождания

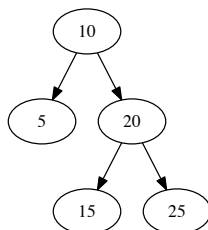
Възел на двоично дърво представяме със следния шаблон на структура:

```
template <class T>
struct BinTreeNode
{
    T data;
    BinTreeNode *left, *right;
    //... помощни конструктори
};
```

Освен ако не е указано друго, задачите по-долу да се решат като се реализират методи на клас `BinTree` със следния скелет:

```
template <class T>
class BinTree
{
    //...
private:
    BinTreeNode<T> *root;
};
```

- 19.1. Да се дефинира метод `count` на клас `BinTree`, който намира броя на елементите на дървото.
- 19.2. Да се дефинира метод `countEvens` на клас `BinTree`, който намира броя на елементите на дърво от числа, които са четни.
- 19.3. Да се дефинира метод `int BTree<T>::searchCount (bool (*pred)(const T&))` към клас `BinTree`, който намира броя на елементите на дървото, които удовлетворяват предиката `pred`.
Да се приложи `searchCount` за решаване на горните две задачи.
- 19.4. Да се дефинира метод `bool BinTree<T>::height ()`, намиращ височината на дърво. *Височина на дърво наричаме дължината (в брой върхове) на най-дългия път от корена до кое да е листо на дървото. Пример. Височината на дървото на Фигура 3.3 е 3.*
- 19.5. Да се дефинира метод `countLeaves` на клас `BinTree`, който намира броя на листата в дървото.



Фигура 3.3: Двоично наредено дърво

- 19.6. Да се дефинира метод `maxLeaf` на клас `BinTree`, който намира най-голямото по стойност листо на непразно дърво. Да се приеме, че за типа `T` на шаблона `BinTree` е дефиниран операторът `<`.
- 19.7. Нека е дадено дървото `t` и низът `s`, съставен само от символите 'L' и 'R' ($s \in \{L, R\}^*$). Нека дефинираме "съответен елемент" на низа `s` в дървото `t` по следния начин:
- Ако дървото `t` е празно, низът `s` няма съответен елемент
 - Ако низът `s` е празен, а дървото `t` - не, то коренът на дървото `t` е съответният елемент на низа `s`
 - Ако първият символ на низа `s` е 'L' и дървото `t` не е празно, то съответният елемент на низа `s` в дървото `t` е съответният елемент на низа `s + 1` в **лявото** поддърво на `t`
 - Ако първият символ на низа `s` е 'R' и дървото `t` не е празно, то съответният елемент на низа `s` в дървото `t` е съответният елемент на низа `s + 1` в **дясното** поддърво на `t`

Пример. За дървото от Фигура 3.3, съответният елемент на празния низ е 10, на низа "RL" е 15, а "RLR" няма съответен елемент.

Да се дефинира метод `T& BinTree<T>::getElement (const char *s)`, който намира съответния елемент на низа `s`. Какво връща методът в случаите на липса на съответен елемент е без значение.

- 19.8. Да се дефинира функция `bool isOrdered(root)`, която по указател към корена на двоично дърво проверява дали дървото е двоично наредено.
Упътване: използвайте помощна функция

19.2 Извеждане и сериализация

В условията по-долу с [подходящ тип] е указано, че част от задачата е да се изберат правилните типове на параметрите на функциите така, че рекурсия да може да се осъществи.

- 19.9. Да се дефинира функция `prettyPrint ([подходящ тип] t)`, отпечатваща двоично дърво на стандартния изход по следния начин: (1) всеки наследник е вдясно от родителя си, (2) елементите на еднакво ниво в дървото се отпечатват на еднаква колона от екрана, (3) десните наследници са на предишен ред от родителя си и (4) левите наследници са следващ ред спрямо родителя си.

Например, дървото от Фигура 3.3 би изглеждало по следния начин (да се отпечатат и номерата на редовете на стандартния изход):

```
1:      25
2:    20
3:      15
4:  10
5:    5
```

Представяне на двоично дърво в “Scheme формат” наричаме еднозначно текстово представяне на структурата от данни, образувано по следните правила:

- Празното дърво се представя с низа “()”
- Нека е дадено дървото t с корен x , ляво поддърво t_l и дясно поддърво t_r . Ако s_l е представянето в “Scheme формат” формат на t_l , а s_r е представянето в “Scheme формат” на t_r , то низът “(x s_l s_r)” е представянето на дървото t , където “ x ”, “ s_l ” и “ s_r ” са съответните низове.

Например, дървото от Фигура 3.3 се представя по следния начин:

```
(10 (20 (25 () ())) (15 () ())) (5 () ()))
```

- 19.10. Да се дефинират функции за сериализация и де-сериализация на двоично дърво, съдържащо числа, като се използва “Scheme формат”.

Може ли да се сериализира дърво, съдържащо други дървета?

Представяне на двоично дърво в “Haskell формат” наричаме еднозначно текстово представяне на структурата от данни, образувано по следните правила:

- Празното дърво се представя с низа “Empty”
- Нека е дадено дървото t с корен x , ляво поддърво t_l и дясно поддърво t_r . Ако s_l е представянето в “Scheme формат” на t_l , а s_r е представянето в “Haskell формат” на t_r , то низът “Node x s_l s_r)” е представянето на дървото t , където “ x ”, “ s_l ” и “ s_r ” са съответните низове.

Например, дървото от Фигура 3.3 се представя по следния начин:

```
Node 10
  Node 20
    Node 25 Empty Empty
    Node 15 Empty Empty
  Node 5 Empty Empty
```

- 19.11. Да се дефинират функции за сериализация и де-сериализация на двоично дърво, съдържащо числа, като се използва “Haskell формат”.

Може ли да се сериализира дърво, съдържащо други дървета?

19.3 Обхождания

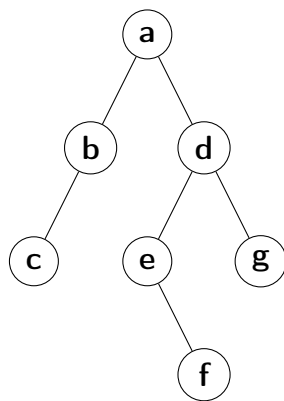
- 19.12. Да се реализира функция `std::vector<T> listLeaves ([подходящ тип]t)` намираща списък със стойностите на листата на дървото.
- 19.13. Да се дефинира функция `std::string findTrace ([подходящ тип]t, const T& x)`. Ако x е елемент на дървото, функцията да връща следата на x (според дефиницията на “следа”, обсъдена на лекции). Ако x не е елемент на дървото, функцията да връща низа “_”.

Пример: За дървото от Фигура 3.3, следата на елемента със стойност 25 е “RR”.

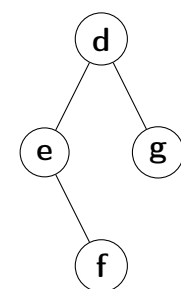
- 19.14. Да се дефинира функция `T getAt([подходящ тип]t, size_t i)`, която намира i -тият пореден елемент на дървото при обхождане корен-ляво-дясно.

Пример: За дървото от Фигура 3.3, елементът с пореден номер 0 е 10, с номер 1 е 5, с номер 2 е 20 и т.н.

- 19.15. Нека е дадено двоично дърво, чиито възли съдържат символи (`char`). Да се намери:



Фигура 3.4: Дърво със символи



Фигура 3.5: Поддърво на дървото от фигура 3.4

- а) Най-дългия символен низ, който може да се получи при последователно конкатениране на символите от корена на дървото по пътя до някое негово листо. На дървото от фигура 3.4 това е низа “adef”.
 - б) Най-дългия символен низ, който може да се получи при конкатениране на символите по възлите на някое ниво в дървото, отляво надясно. На дървото от фигура 3.4 това е низа “seg”.
- 19.16. Нека са дадени две дървета t_1 и t_2 . Да се напише функция `bool equals([подходящ тип]  $t_1$ , [подходящ тип]  $t_2$ )`, която проверява дали двете дървета са еднакви (използвайте интуитивна дефиниция за еднаквост на дървета).
- 19.17. Нека са дадени две дървета t_1 и t_2 . Да се напише функция `bool subtree([подходящ тип]  $t_1$ , [подходящ тип]  $t_2$ )`, която проверява дали някое поддърво на t_1 е еднакво с t_2 . Вж. фигури 3.4 и 3.5.

19.4 Представяне на аритметичен израз чрез дърво

- 19.18. Нека е даден израз, построен по правилата на следната граматика:

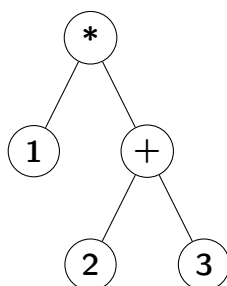
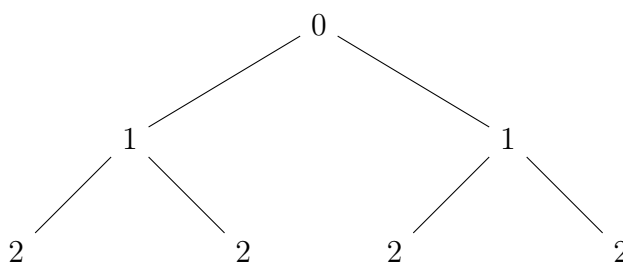
```
<expression> ::= <digit> | (<expression><operator><expression>)
<digit> ::= 1..9
<operator> ::= + | - | * | /
```

Да се реализира функция на клас `BinTree<char> parseExpression (std::string s)`, която по правилно построен израз, записан в низа `s`, създава двоично дърво от символи, представящо израза по следното правило:

- Ако изразът е от типа “`x`”, където `x` е цифра, то съответното му дърво е листо със стойност символа `x`.
- Ако изразът е от типа “(`<израз 1><op><израз 2>`)” , то съответното му дърво има като стойност на корена символа на съответния оператор, ляво поддърво, съответно на `<израз 1>` и дясно поддърво, съответно на `<израз 2>`.

Дървото на Фигура 3.10 съответства на израза `(1*(2+3))`.

- 19.19. Да се реализира функция `double calculateExpresisonTree ([подходящ тип]  $t$ )`, която намира стойността на израз по дадено дърво, построено от решението на предишната задача.

Фигура 3.6: Дърво на израза $(1*(2+3))$ 

Фигура 3.7: Идеално балансирано двоично дърво с височина 3

19.5 Построяване и модификации на дърво

19.20. По дадено число h да се построи идеално балансирано двоично дърво с височина h . Стойността на всеки от елементите на дървото да е равна на нивото, в което се намира елемента (вж. Фигура 3.7).

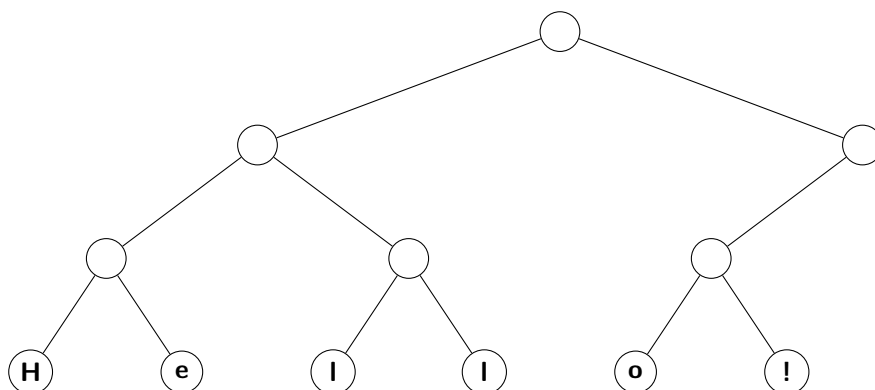
19.21. Нека е даден символен низ s с дължина n . Нека h е такова, че $2^h \geq n > 2^{h-1}$ (минималната височина на двоично дърво, което има поне n листа в последното си ниво). Да се дефинира функция, която по даден низ s построява двоично дърво от символи с височина h , такова, че низът s е разположен в листата на дървото, четени от ляво надясно. Възлите на дървото, които не са листа, да съдържат символа интервал. Вж. Фигура 3.8

19.22. Стойността на всеки възел V в дадено двоично дърво от числа да се замени с броя на всички елементи на поддървото, на което V е корен. Вж. Фигура 3.9.

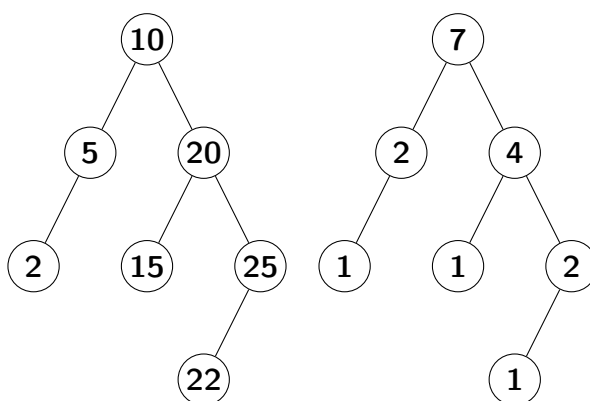
При операцията всеки от възлите да бъде посетен най-много веднъж!

19.23. Дадено е дърво с низове по върховете. Дървото да се балансира по следния начин:

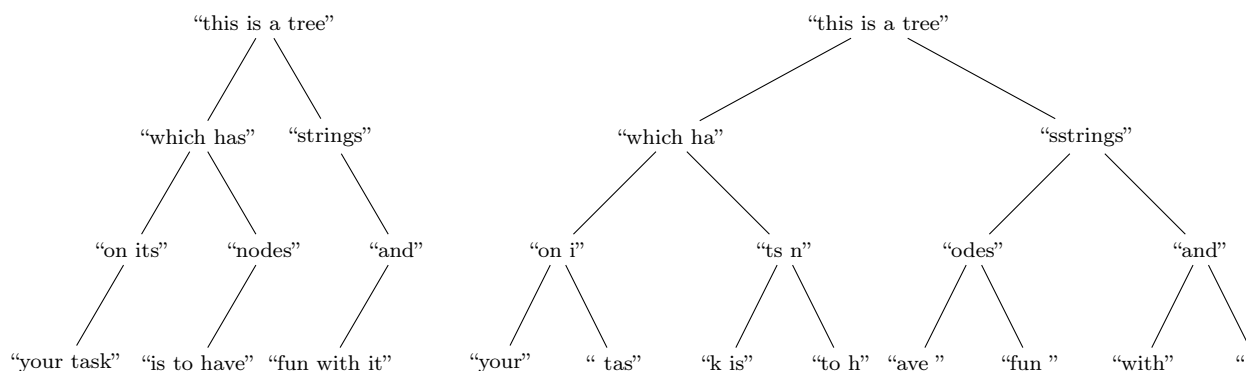
а) Резултатното дърво има същия брой нива като изходното.



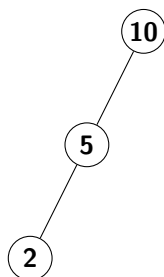
Фигура 3.8: Дърво, в чиито листа е разположен низът “Hello!”



Фигура 3.9: Примерно дърво и същото дърво, стойностите на чиито възли са замесени с размера на съответното им поддърво



Фигура 3.10: Примерно дърво от низове преди и след балансирането



Фигура 3.11: “Орязана” версия на дървот от фигура 3.9

- б) Всяко k -то ниво на резултатното дърво да съдържа точно 2^k елемента (считаме, че коренът е на ниво 0).
- в) Нека s_k е низът, получен при конкатенацията на всички низове на ниво k на изходното дърво, обхождани от ляво надясно. Нека дължината на низа s_k е n_k символа. i -тият пореден елемент на нивото k в резултатното дърво да съдържа i -тата поредна последнователност от $\lceil n_k/2^k \rceil$ на брой символи на s_k , освен най-десния, който съдържа последните “останали” символи от s_k . Т.е. s_k да се “раздели” поравно между елементите в резултатното дърво.

Ако $n_k < 2^k$, да се използва низ, получен от минималния брой копия на s_k , така, че да се получи низ с дължина $\geq 2^k$.

На Фигура 3.10 са илюстрирани примерно изходно дърво и резултатът от балансирането му по горното правило. Всички елементи на ниво 1, освен последния, съдържат по $8 = \lceil 16/2 \rceil$ символа. Всички елементи на ниво 2, освен последния, съдържат по $4 = \lceil 14/4 \rceil$ символа и т.н.

Упътване: предварително намерете вектора $(s_0, s_1, \dots, s_{h-1})$ и го използвайте за балансирането.

- 19.24. Да се дефинира функция **prunetree**, която “отсича” (премахва) всички поддървета на дадено двоично дърво, чиито корени имат стойност, по-малки от стойността на някой от наследниците им. Дървото от фигура 3.11 се получава при “орязване” на дървото от фигура 3.9.

20 Префиксно дърво (Trie)

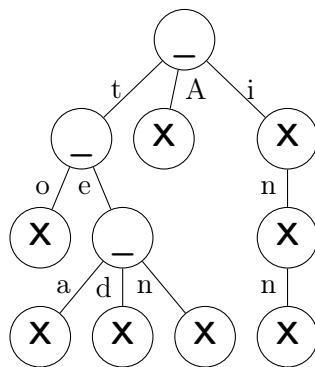
Префиксното дърво (Trie) може да се използва за представяне на множество от думи (вж. Фигура 3.12). Възлите в дървото може да имат произволен брой наследници, като всяка дъга е аотирана със символ. Всеки възел е аотиран с флаг, който разделя възлите на финални и нефинални (на схемата финалните възли са отбелязани с “X”). Символите по дъгите на всеки път от корена на дървото до финален възел задават една дума от множеството. Дадена дума е елемент на множеството т.с.т.к. съществува такъв път. Обърнете внимание, че даден възел може да е както финален, така и част от по-дълга дума. Това се случва, когато в множеството се съдържа някоя дума заедно с някой нейн префикс. Използваме просто представяне на възел в дървото, като сме ограничили множеството на допустимите символи до малки латински букви:

```
struct TrieNode
{
    bool eow;
    TrieNode *children[26];
    TrieNode ();
};
```

На лекции дефинираме функция за добавяне на дума към множеството, представено с дадено дърво и за извеждане на дърво в dot формат:

```
void trieInsert (TrieNode *&root, const char *s);
void dotPrint (std::ostream& out, TrieNode *root)
```

- 20.1. Да се напише функция за намиране на височина на Trie.
- 20.2. Да се напише функция за намиране на дължината на най-дългата дума, записана в Trie.
- 20.3. Да се напише функция за намиране на броя на записаните думи в Trie.
- 20.4. Да се напише функция за намиране на броя на буквите на латинската азбука, които *не участват* в никоя дума в дадено Trie.
- 20.5. Чрез Trie да се реализира клас **StringSet**, представящ множество от символни низове. За класа да се реализират:
 - Канонични методи
 - Оператор == за сравнение на две множества



Фигура 3.12: Trie, съдържащо думите “to”, “tea”, “ted”, “ten”, “A”, “i”, “in” и “inn”

- Оператори + и += за добавяне на дума към множеството
- Булев оператор [], проверяващ дали дадена дума се съдържа в множеството
- Оператори - и -= за премахване на дума от множеството
- Оператори < и <=, реализиращи проверка за подмножество

20.6. За клас **StringSet** да се реализира итератор, позволяващ обхождане на думите в множеството в реда на лексикографската наредба.

21 Изображения (Maps)

21.1 HashMap

Следните задачи са базирани на проста реализация на хеш таблица с отворено адресиране:

```
template <class KeyType, class ValueType>
class HashMap;
```

Редовете в хеш таблицата са елементи от структурата:

```
template <class KeyType, class ValueType>
struct TableElement
{
    KeyType key;
    ValueType value;
    TableElement<KeyType,ValueType> *next;
}
```

Самата хеш таблица е представена с член данната на класа `HashMap TableElement<KeyType, ValueType> **table`.

Реализиран е и итератор, позволяващ обхождане на ключовете, записани в хеш таблицата.

- 21.1. Да се дефинира метод `HashMap::efficiency()`, който изчислява ефективността на хеш таблицата като отношението $\frac{all-coliding}{all}$, където *coliding* е броят на ключовете, записани при колизия, а *all* е броят на всички записани ключове.
- 21.2. Да се дефинира оператор `<<` за клас `HashMap`, който отпечатва в поток всички двойки ключ-стойност в Хеш таблицата.
- 21.3. Да се напише програма, която въвежда от клавиатурата две текста с произволна големина t_1 и t_2 . Програмата да извежда броя на всички срещания на думи в t_2 , които се срещат и в t_1 .

Пример: за следните текстове

In computing, a hash table (hash map) is a data structure used to implement an associative array, a structure that can map keys to values. A hash table uses a hash function to compute an index into an array of buckets or slots, from which the correct value can be found.

и

Ideally, the hash function will assign each key to a unique bucket, but this situation is rarely achievable in practice (usually some keys will hash to the same bucket)

Този брой е 10, съставен от думите *the* (2 срещания във втория текст), *a* (1 срещане), *hash* (2), *function* (1), *to* (2), *is* (1), *keys* (1).

- 21.4. Да се напише програма, която въвежда от клавиатурата две текста с произволна големина t_1 и t_2 . Програмата да извежда броя на уникалните думи в t_2 , които се срещат и в t_1 .

Пример: за двата текста от предишната задача, този брой е 7, съставен от думите *the*, *a*, *hash*, *function*, *to*, *is*, *keys*.

- 21.5. Да се напише програма, която прочита от входа даден текст с произволна големина и намира такава дума с дължина повече от 3 букви, която се среща най-често в текста. Пример: за текста

In computing, a hash table (hash map) is a data structure used to implement an associative array, a structure that can map keys to values. A hash table uses a hash function to compute an index into an array of buckets or slots, from which the correct value can be found.

Най-често срещаната дума е *hash*.

- 21.6. От клавиатурата да се въведе цялото положително число n , следвано от $2 \times n$ цели положителни числа $a_1, b_1, a_2, b_2, \dots, a_n, b_n$. Програмата да печата на екрана "Yes", ако изображението, дефинирано като $h(a_i) = b_i, i = 1, \dots, n$ е добре дефинирана функция. Т.е. програмата да проверява дали има два различни индекса i и j , за които е изпълнено $a_i = a_j$, но $b_i \neq b_j$.

- 21.7. Да се дефинира метод

```
void map ([подходящ тип] f)
```

на хеш-таблицата, който прилага функцията f над всички стойности в хеш-таблицата.

- 21.8. Да се добави възможност за итериране само на тези ключове от хеш таблицата, чиито стойности удовлетворяват някакъв предикат, зададен чрез (шаблон на) указател към функция. *Упътване: разширете итератора така, че да може да се инициализира с предикат. Предикатът може да се задава, например, чрез метода begin. Последният трябва да има стойност по подразбиране на параметъра си, за да може да се ползва цикъл for (<key> : <hash table>).*

Глава 4

Функционално програмиране

22 Уводни задачи

- 22.1. Да се дефинира функция, която намира лицето на триъгълник по дадени: а) дължини на страна и височина към нея; б) три страни.
- 22.2. Да се дефинира функция, която по двойка (x, y) кординати на точка от равнината намира на кой квадрант принадлежи точката. Да се разгледат случаите, когато точката принадлежи на някоя от координатните оси или съвпада с центъра на координатната система.
- 22.3. Да се дефинира функция, която има стойност истина, ако посоченото условие е вярно и стойност - лъжа, в противен случай:
- а) цялото число p се дели на 4 или на 7;
 - б) уравнението $ax^2 + bx + c = 0 (a \neq 0)$ няма реални корени;
 - в) точка с координати (a, b) лежи във вътрешността на кръг с радиус 5 и център $(0, 1)$; г) точка с координати (a, b) лежи извън кръга с център (c, d) и радиус f ;
 - г) точка принадлежи на частта от кръга с център $(0, 0)$ и радиус 5 в трети квадрант;
 - д) точка принадлежи на венеца с център $(0, 0)$ и радиуси 5 и 10;
 - е) x принадлежи на отсечката $[0, 1]$;
 - ж) x е равно на $\max \{a, b, c\}$;
 - з) x е различно от $\max \{a, b, c\}$;
 - и) нито едно от числата a, b и c не е положително;
 - к) цифрата 7 влиза в запис на положителното трицифрено число p ;
 - л) цифрите на трицифреното число m са различни;

- м) поне две от цифрите на трицифреното число m са равни помежду си;
 - н) цифрите на трицифреното естествено число x образуват строго растяща или строго намаляваща редица;
 - о) десетичните записи на трицифрените естествени числа x и y са симетрични;
 - п) естественото число x , за което се знае, че е по-малко от 23, е просто.
- 22.4. Да се напише функция, която намира броя на цифрите в десетичния запис на дадено естествено число.
- 22.5. Да се напише функция, която намира сумата на цифрите в десетичния запис на дадено естествено число.
- 22.6. Да се дефинира функцията $pow(x, k) = x^k$ за цели положителни числа x и k .
- 22.7. Да се напише функция, която по дадени естествено число x и едноцифрено число k намира а) дали k се среща в десетичния запис на x и б) колко пъти k се среща в десетичния запис на x .
- 22.8. Да се напише функция, която проверява дали дадена година е високосна. (вж. задача 2.12. в [1])
- 22.9. Да се напише функция, която по дадено естествено число n ($n \geq 1$) намира броя на тези елементи от серията числа $i^3 + 13 \times i \times n + n^3$, $i = 1, 2, \dots, n$, които са кратни на 5 или на 9.
- 22.10. Да се дефинира функция, която по естествени числа n и k намира дали n е точна степен на числото k .
- Упътване: Разделете променливата n на променливата k “колкото пъти е възможно” и проверете дали n достига единица или някое друго число след края на процеса. Използвайте оператора за намиране на остатък при целочислено деление и оператора за целочислено деление.*
- 22.11. Едно положително цяло число е свършено, ако е равно на сумата от своите делители (без самото число). Например, 6 е свършено, защото $6 = 1+2+3$; числото 1 не е свършено. Да се дефинира функция, която проверява дали дадено положително цяло число е свършено.

23 Проста рекурсия над списъци

Долните задачи да се решат с помощта на проста рекурсия и функции и като `null`, `length`, `head`, `tail` като упражнение за рекурсия и употреба на базосите функции за работа със списъци.

- 23.1. Да се реализира собствен вариант на функциите `length`, `elem` и `sum` за намиране на дължина на списък, проверка за принадлежност на елемент към списък и намиране на сумата на елементите на списък от числа.
- 23.2. Да се реализира функция `count x l`, която преброява колко пъти се среща елемента `x` в `l`.
- 23.3. Да се реализира функция `index x l`, която намира поредния номер на първото срещане на елемента `x` в `l`. Например `index 7 [1,2,7,3,2] -> 3`.
- 23.4. Да се реализира функция `sublist l1 l2`, която проверява дали всички елементи на `l1` са елементи и на `l2`.
- 23.5. Да се реализира функция `common l1 l2`, която преброява колко от елементите на `l1` са елементи и на `l2`.
- 23.6. Да се реализира функция `duplicates l`, която проверява дали в списъка `l` има повтарящи се елементи.

24 Конструирание на списъци

- 24.1. Да се съставят следните списъци:

- а) Първите n четни числа;
- б) Първите n члена на аритметична прогресия с първи член a и разлика d ;
- в) $[1!, 2!, \dots, n!]$ за дадено n ;
- г) Всички четни числа;
- д) Всички членове на аритметична прогресия с първи член a и разлика d ;
- е) $[1!, 2!, \dots]$ (безкраен списък)

- 24.2. Да се дефинира функция, която по дадено естествено число n връща списък с цифрите му, четени отдясно на ляво.
- 24.3. (*) Да се дефинира функция, която по дадено естествено число n връща списък с цифрите му, четени отдясно на ляво, без повторения на елементите на списъка.

- 24.4. Едно положително цяло число е съвършено, ако е равно на сумата от своите делители (без самото число). Например, 6 е съвършено, защото $6 = 1+2+3$; числото 1 не е съвършено. Да се дефинира функция, която създава списък с всички съвършени числа, ненадминаващи дадено положително цяло число в параметър n .
- 24.5. Да се дефинира функция *histogram*, която по символен низ s връща списък от двойки (c_i, n_i) , където c_i са различните символи от s , а n_i е броя на срещания на c_i в s . Например, *histogram* "abracadabra" $\rightarrow [(a, 5), (b, 2), (r, 2), (c, 1), (d, 1)]$. Използвайте помощни функции.

25 Работа със списъци и образци

- 25.1. Дефинирайте функция *say*, която по едноцифрено цяло число връща неговото наименование. Например, *say* 3 $\rightarrow$ "three".
- 25.2. Да се дефинира функция, която по два списъка намира дължината на най-дългия им общ префикс.
- 25.3. Да се дефинира функция *countEvenOddl*, която за списъка от цели числа l връща наредена двойка от броя на четните и броя на нечетните числа в l .
- 25.4. Да се дефинира функция *pivotlx*, която за списъка от числа l и числото x връща наредена двойка (l_1, l_2) , където l_1 е списък от елементите на l , по-малки от x , а l_2 е списък от елементите на l , по-големи или равни на x .
- 25.5. Да се дефинира функция, която в даден низ замества всички малки латински букви със съответните им големи латински букви.
- 25.6. Да се дефинира функция, която проверява дали даден низ е палиндром, т.е. дали се е еднакъв при четене от ляво на дясно и от дясно на ляво.
- 25.7. Да се дефинира функция, която по даден списък от цели числа l връща списък от всички двойки (a, b) от l , за които a и b са съседни елементи в l и $a < b$.
- 25.8. Да се дефинира функция *groupsof* 1 x , която разделя списъка l на групи от по x елемента.
Например, *groupsof* [1, 2, 3, 4, 5, 6, 7, 8] 3 $\rightarrow$ [[1, 2, 3], [4, 5, 6], [7, 8]].
- 25.9. Да се дефинира функция *flattenl* :: $[[a]] \rightarrow [a]$, която получава списък от списъци и връща списък, който съдържа всички елементи на входните списъци.

- 25.10. Да се дефинира функцията $decode\ l :: [(Int, a)] \rightarrow [a]$, която получава списък от двойки и връща списък, който съдържа всички елементи на входния списък, като всеки елемент се повтаря толкова пъти, колкото е указано в първия елемент на двойката.
- 25.11. Да се дефинира функцията $pack\ l :: [a] \rightarrow [[a]]$, която получава списък и връща списък от списъци, като всеки от тях съдържа всички последователни еднакви елементи на входния списък.
- Например, $pack\ [1, 1, 1, 2, 2, 3, 4, 4, 4, 4] \rightarrow [[1, 1, 1], [2, 2], [3], [4, 4, 4, 4]]$.
- 25.12. Да се дефинира функцията $encode\ l :: [a] \rightarrow [(Int, a)]$, която получава списък и връща списък от двойки, където първият елемент на двойката е броят на последователните еднакви елементи от входния списък, а вторият елемент е самият елемент.
- Например, $encode\ [1, 1, 1, 2, 2, 3, 4, 4, 4, 4] \rightarrow [(3, 1), (2, 2), (1, 3), (4, 4)]$.
- 25.13. Да се дефинира функцията `remove x l`, която премахва (а) първото срещане на елемента x от списъка l и (б) всички срещания на елемента x от списъка l .
- 25.14. Да се дефинира функцията `removeDuplicates l`, която премахва всички повторения на елементите на списъка l .
- 25.15. За даден списък L , да се намерят елементите на списъка, чиято стойност е по-голяма от сумата на предхождащите ги елементи. Пример $[1, 2, 5, 9, 16] \rightarrow [1, 2, 5, 9]$.
- 25.16. Да се дефинира функцията `mergeevenodd l1 l2`, която получава два списъка от цели числа $l1$ и $l2$ и връща списък, чиито елементи на четни позиции са елементите на $l1$, а тези на нечетни позиции са елементите на $l2$. Пример: `mergeevenodd [1,2,3] [4,5,6] -> [1,4,2,5,3,6]`.

26 Дефиниране на алгебрични типове

- 26.1. Да се дефинира тип `DayOfWeek`, представящ ден от седмицата. За типа да се реализират възможности за:

- Въвеждане и извеждане
- Сравнение на дни с `==, <, >`
- Намиране на следващ и предишен ден от седмицата
- Преобразуване от и до число
- Създаване на интервал от дни

Съответните възможности да се добавят по два начина:

- Чрез нарочни функции за целта
- Чрез наследяване на базови класове

26.2. В един университет има студенти и преподаватели. Всеки студент има факултетен номер, име и списък от имената на курсове, които посещава. Всеки преподавател има име, списък от имената на курсове, които преподава и номер на кабинет. Да се дефинира алгебричен тип **Member** със съответни конструктори за студент и преподавател.

- По списък **[Member]** да се намери броя на студентите.
- Да се дефинира функция **attendees :: String -> [Member] -> Int**, която по име на предмет и списък от членове на университета намира броя на студентите, които слушат предмета.
- Да се дефинира функция **classmembers :: String -> [Member] -> [Member]**, която по име на предмет и списък от членове на университета намира списък само на тези членове, които преподават или слушат дадения предмет.
- Да се дефинира функция **namesitles :: [Member] -> [String]**, която връща имената на всички членове на университета, като пред името на всеки преподавател автоматично добавя титлата “проф.”.
- (*) Да се дефинира функция **bussiest :: [Member] -> Int**, която намира кабинета с най-много преподаватели.

27 Монадът Maybe

Везните, подобни на показаната на фигура 27, служат за сравнение на тежести, поставени в лявата и дясната теглилка на везната. Рамото на везната се накланя в посоката на по-голямата тежест. Такива везни могат да се повредят, ако разликата между тежестите е твърде голяма, което би довело до преобръщане на везната, или някоя от тежестите е твърде голяма сама по себе си, което би довело до счупване на теглилката.



Фигура 27: Везна

“Конфигурация” на везната наричаме двойката числа, задаващи големината на тежестите в лявата и дясната теглилка. Конфигурацията наричаме “валидна”, ако няма да доведе до повреда на везната и “невалидна” в противен случай.

Дефинирайте подходящ тип **Scale** за представяне на конфигурацията на везна.

- 27.1. Да се дефинират функции над конфигурацията на везна, чрез които може да се доавя тежест в лявата или дясната теглилка.
- 27.2. Функциите за добавяне на тежести да използват монада **Maybe** за третиране на невалидни конфигурации. Считаме, че везната е в невалидна конфигурация, ако някоя от тежестите е над 100 или разликата на тежестите е над 50.
- 27.3. Да се илюстрира няколкократно добавяне на тежести чрез приложение на оператора `>>=`.
- 27.4. Да се дефинира функция **heavier**, която по конфигурация на везна връща по-голямата от двете тежести или нищо, ако тежестите са еднакви.
Пример: Следният израз да има оценка `Just 16`:

```
Just $ Scale 5 3 >>= left 10 >>= right 10 >>= right 3 >>= heavier
```

28 Функции от по-висок ред

- 28.1. Направете и тествайте собствена реализация на функциите **map**, **filter** и **fold(l,r,l1,r1)**.
- 28.2. Нека е даден списък $l :: [(Int, Int, Int)]$ с тройки (a_i, b_i, c_i) . С помощта на **map**, **fold** и **filter** да се намерят:
 - а) Списъка от сумите на елементите на тройките $[(a_i + b_i + c_i)]$
 - б) Тройка от сумите на отделните компоненти на елементите на **l**, $(\sum a_i, \sum b_i, \sum c_i)$
 - в) Броя на тройките, за които $a_i + b_i > c_i$
 - г) Дали има поне една тройка, за която $a_i = b_i = c_i$ (**True** или **False**)
- 28.3. За списък от числа L да се намери списък с само с тези числа, които съвпадат с поредния си номер в L . Например $[1, 5, 3, 4, 2] \rightarrow [1, 3, 4]$.
- 28.4. За списък от числа L да се намери списък със сумите на всички двойки последователни елементи на L . Например $[1, 5, 3, 4, 2] \rightarrow [6, 8, 7, 6]$. *Упътване: Използвайте **zip**.*

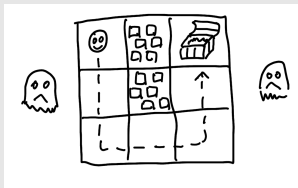
- 28.5. С помощта на `zipWith` да се дефинира функция `sums :: [Int] -> [Int]`, която по списък от числа $L = l_1, l_2, l_3, \dots$ намира списъка $S = l_1, (l_1 + l_2), (l_1 + l_2 + l_3), \dots$.
- 28.6. Да се дефинира функция `separate :: (a->Bool) -> [a] -> ([a], [a])`, която по предикат p и списък l връща двойката $(\text{pref}, \text{suf})$. `pref` е най-дългият възможен префикс, такъв че всички негови елементи удовлетворяват p . `suf` е останалата част от списъка l . Например, `separate even [2,4,6,7,8,10] -> ([2,4,6], [7,8,10])`. *Забележка: вижте функцията `break` в `Prelude`.*
- 28.7. Да се дефинира функция `split :: (a->Bool) -> [a] -> [[a]]`, получаваща предикат p и списък l . Елементите на l , удовлетворяващи p , се считат за “разделители” в l и списъкът се разделя на части, обособени от тези разделители. Например:

```
split (==',') "part1 , part_2 , part3"
  -> ["part1", "part_2", "part3"]
```

На лекции разглеждаме модел на лабиринт, съдържащ цел и препятствия.

```
type Pos = (Int, Int)

data Tile = Road | Wall | Gold
          deriving (Show, Eq)
data Game = Game { pos :: Pos, world :: [[Tile]] }
          deriving (Show)
myWorld :: Game = Game { pos = (0, 0),
                        world = [[Road, Wall, Gold],
                                [Road, Wall, Road],
                                [Road, Road, Road]] }
```



Следите функции “придвижват” играча в лабиринта:

```
left (Game (x, y) w) = Game (x-1, y) w
right (Game (x, y) w) = Game (x+1, y) w
up (Game (x, y) w) = Game (x, y-1) w
```

```
down (Game (x, y) w) = Game (x, y+1) w
```

И помощни функции при изследване на играта:

```
foundGold :: Game -> Bool
foundGold (Game (x,y) w) = (w !! y) !! x == Gold
stuck :: Game -> Bool
stuck (Game (x,y) w) = dead w (x, y)
dead :: [[Tile]] -> Pos -> Bool
dead w (x, y) = x < 0 || x >= length w ||
                y < 0 || y >= length w ||
                (w !! y) !! x == Wall
move :: Game -> (Pos -> Pos) -> Maybe Game
move (Game p w) mv = if dead w (mv p)
                        then Nothing
                        else Just $ Game (mv p) w
```

28.8. С помощта на `map`, `mapMaybe`, `filter` и `fold` да се намерят:

- а) Броя на препятствията в даден лабиринт
- б) Броя съседи на целта, които не са препятствия
- в) Дали целта е оградена изцяло от препятствия (`True` или `False`)

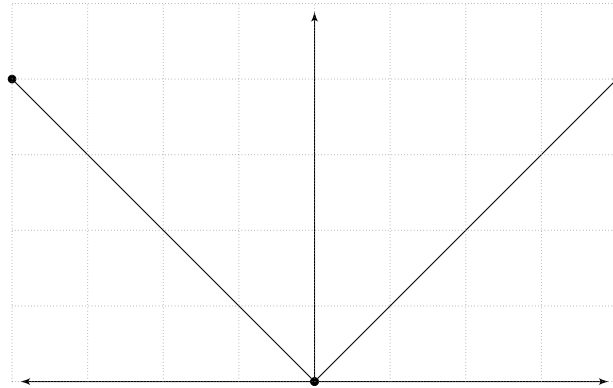
28.9. (*) С помощта на `foldr` да се дефинира функция, която проверява дали дали даден списък от числа `l :: [Int]` е нареден във възходящ ред.
Упътване: Използвайте двойка (`Bool, Int`) за акумулатор.

28.10. За даден списък `l = [a1, ..., an] :: [a]` и функция `p : a -> a`, чрез еднократно приложение на `foldr` да се “разбие” списъка `l` на подсписъци, за елементите на които `p` има еднаква стойност. Т.е. да се построи списък от списъци от числа `G = [G1, ..., Gk] :: [[a]]`, като елементите на `G` са съставени от елементите на `l` и за и за всяко `Gi`, `p(x) == p(y)` за всички `x, y ∈ Gi`. Операцията е сходна с операцията `Group by` известна от базите данни, която групира данните спрямо някакъв техник атрибут.

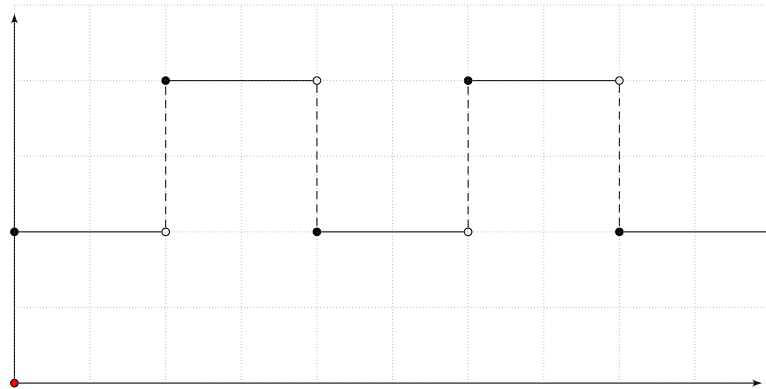
Пример: за списъка `l = [1..10]` и функцията `p(x) = mod x 3`, ще получим групите `[[3, 6, 9], [1, 7, 10], [2, 8]]`.

29 λ функции

29.1. “Двойно-линейна функция” наричаме функция `f : Double -> Double`, която има стойност `a1x` за `x ≤ 0` и стойност `a2x` за `x > 0`. Да се дефинира функция `dlin :: Double -> Double -> (Double -> Double)`, която по аргументи `a1` и `a2`, създава съответната двойно-линейна функция. Функцията на фигура 4.1 може да се създаде чрез `dlin (-1) 1`.



Фигура 4.1: Двойно-линейна функция за $a_1 = -1$, $a_2 = 1$



Фигура 4.2: Правоъгълна функция, $low = 2$, $high = 4$, $step = 2$

29.2. “Правоъгълна функция” наричаме функция $f : Double \rightarrow Double$, която редува две стойности low и $high$, $low < high$, като ги разменя със стъпка $step$ по аргумента. Вж. фиг 4.2. Да се дефинира функция `rectfn :: Double -> Double -> Double -> (Double->Double)`, която по аргументи low , $high$ и $step$ създава съответната правоъгълна функция. Функцията на фигурата може да се създаде чрез `rectfn 2 4 2`.

29.3. Да се дефинира функция `createfn :: [(a,b)]->(a->b)`, която по даден списък от двойки (a_i, b_i) , връща функция $f : a \rightarrow b$ дефинирана за всички a_i , така че $f(a_i) = b_i$. Например:

```
f = createfn [(1,2) ,(2,4) ,(3,6) ]
f 2 -> 4
```

29.4. Да се дефинира функция `caseof :: [(a->Bool,a->b)] -> (a -> b)`. Функцията приема k на брой двойки (p_i, f_i) , където p_i е предикат над a , а f_i е функция $a \rightarrow b$. Функцията `caseof` връща функция, която приема

аргумент x от тип a и връща стойността $f_i(x)$ на първата функция в списъка, за която предикатът $p_i(x)$ е изпълнен. Ако няма такъв предикат, да се връща стойността на последната функция в списъка. Например, функцията `myf`, дефинирана така:

```
myf = caseof [(even, \x -> x+1), (odd, \x -> x-1)]
```

ще увеличава с единица четните числа и ще намалява с единица нечетните числа.

- 29.5. Да се дефинира функция `member :: [a] -> (a -> bool)`. Стойността на функцията за даден списък l е функция над a , която проверява дали a е елемент на l . На следния пример функцията `myf` има стойност истина за всички четни числа и лъжа за останалите.

```
myf = member [2, 4, ...]
```

- 29.6. Да се дефинира функция `integrate :: (Double -> Double) -> Int -> Double -> Double -> Double`, която по функция $f : Double \rightarrow Double$ и цяло положително число N връща $i(a, b)$ от тип $i : Double \times Double \rightarrow Double$, изчисляваща приближено стойността на определения интеграл $\int_a^b f(x)dx$ (приемайки, че такъв съществува) по метода на трапеците, по-специално Chained trapezoidal rule[8] с N на брой подинтервала. (вж. фиг. 11.2)

30 Вход/Изход

Пренасочване на стандартния вход и изход

Когато една програма се изпълнява от операционната система, тя получава достъп за четене до поток от данни, наречен “стандартен вход” и достъп за писане до “стандартен изход”. Ако програмата е стартирана в терминална сесия под Posix операционни системи (Mac OS, Linux) или Windows, стандартният вход се състои от поредицата символи, въведени от потребителя чрез клавиатурата. Стандартният изход се визуализира под формата на символи на терминалния прозорец. Това позволява така наречените “конзолни проложения”, които са предвидени да работят в терминал, да реализират диалогов интерфейс с потребителя. На езика C++, стандартният вход и изход са достъпни чрез потоците `cin` и `cout`, дефинирани в `iostream`. На езика Haskell, достъп до стандартният вход и изход се осъществява чрез функции като `getLine`, `putStr` и `putStrLn`.

Операционните системи позволяват стандартният вход да бъде пренасочен така, че вместо от въведените чрез клавиатурата символи, входът да се състои от съдържанието на файл или от изхода на друга програма. Стандартният изход може да бъде пренасочен така, че вместо да се визу-

ализира на терминала, да се запише във файл или да бъде предаден като вход на друга програма.

При Posix операционните системи, пренасочването на входа от файл или изхода към файл става така:

```
./myprog < input.txt  
./myprog > output.txt
```

Съответно, пренасочването на изхода на една програма като вход на друга става така:

```
ls -l | ./myprog
```

При тази команда, изходът на командата `ls` се подава като вход на програмата `myprog`.

Под Windows, пренасочването на входа и изхода от файл става по аналогичен начин:

```
myprog.exe < input.txt  
myprog.exe > output.txt
```

30.1. Да се дефинира програма, която прочита изхода на командата `ls -l` под MacOS и Linux или `dir` под Window и извежда на стандартния изход:

- а) Броят на файловете и броят на поддиректориите в текущата директория
- б) Броят на файловете в текущата директория с разширение `.hs`
- в) Името на най-големия файл в текущата директория
- г) Името на най-новия файл в текущата директория

Форматът CSV

Форматът “Comma Separated Values (CSV)” е текстов формат за представяне на таблични данни. В CSV формат, всеки ред от таблицата се представя като поредица от стойности, разделени със запетаи.

Например:

```
Kalin Georgiev,M,01-01-1981  
Maria Ivanova,F,05-05-2003
```

Ако в рамките на дадена стойност се съдържа символът за запетая, за да не се интерпретира той като разделител, цялата стойност се поставя в двойни кавички. От друга страна, ако искаме самият низ да съдържа двойни кавички, то всяка двойна кавичка в него ограждаме с други две. Например следния низ:

```
"Hello ""world"", have a nice day!"
```

се интерпретира като стойността `Hello "world", have a nice day!`.

30.2. Да се дефинира функция `parseCSV :: String -> [[String]]`, която по даден ред от `csv` файл, представен като символен низ, връща списък от символни низове, представляващи стойностите му, като се разглеждат случаите на стойности, оградени с двойни кавички.

Разгледаната в задача 28.7. функция `split` разделя символния низ прямо даден разделител, в случая запетая. Резултатът от функцията може да се обработи допълнително:

- а) Да се слее в един низ всяка редица от съседни елементи $a_i, a_{i+1}, \dots, a_k$, такава че (1) a_i започва с двойна кавичка; (2) a_k завършва с двойна кавичка; (3)никой от останалите $a_{i+1}, \dots, a_{k-1}$ не започва или завършва с двойна кавичка. *Упражнете се да извършите тази трансформация чрез `fold`.*
- б) се заместят всички срещания на три последователни двойни кавички `"""` с една двойна кавичка `"`.
- в) Функцията да се преработи до `Maybe [[String]]` и да разглежда случаи на грешна употреба на кавички – например две последователни кавички. Кои са всички възможни случаи на грешна употреба на кавички?

31 Индуктивни типове данни

Gantt диаграми

В управлението на проекти, Gantt диаграмите предоставят начин за визуализиране на задачите, от които се състои даден проект, както и за анализ на зависимостите между тях. В тези диаграми всяка задача се характеризира с начало и продължителност. Някои от елементите на диаграмата са “обобщени задачи”, т.е. задачи, които се състоят от други задачи, както прости, така и обобщени.

Работа с дати в Haskell

Модулът `Data.Time.Calendar` дава удобни средства за работа с дати от грегорианския календар. Основният тип `Day` представя конкретна дата от календара. Функции като `addDays` и `diffDays`, както и операторите за сравнение, дават възможност за изчисления с дати и сравнения на дати. Чрез `read` може да се инициализира дата от символен низ във формат `"YYYY-MM-DD"`. Функцията `show` позволява преобразуване на дата към символен низ.

Например, ако `start` е дадена дата в миналото, а `today` е текущата дата, то изразът `(addDays n start) > today` е истина, ако задача, която е започнала на дата `start` и е с продължителност `n` дни, е приключила към днешна дата.

- 31.1. Да се дефинира индуктивен тип `Task`, който представя задача от Gantt диаграма. Задачата може да бъде или проста (`SimpleTask`), или обобщена (`ComplexTask`). Простите задачи се характеризират с име, начало и продължителност. Обобщените задачи се характеризират с име и списък от подзадачи, които могат да бъдат както прости, така и обобщени.

Да се дефинира функция `completed :: Task -> Day -> Bool`, която по задача и дата връща дали задачата е приключила към тази дата.

- 31.2. Да се дефинира функция `duration :: Task -> Int`, която по задача връща нейната продължителност в дни. Продължителността на обобщена задача представлява разликата между дата на започване на най-рано започващата ѝ подзадача и дата на приключване на най-късно приключващата ѝ подзадача.

- 31.3. Да се реализира извеждане на информацията за задача във `xml`-подобен формат. Например:

```
<complex descr="Take_a_course">
  <complex descr="Study">
    <simple descr="Read_Textbook" start="2025-02-01" duration="20"/>
    <simple descr="Do_homeworks" start="2025-02-10" duration="20"/>
  </complex>
  <simple descr="Take_Exam" start="2025-03-01" duration="1"/>
</complex>
```

- 31.4. (*) При извеждането на информацията за задача във `xml`-подобен формат, да се извърши индентация на вложените елементи, както е в примера на задача 31.3..

32 Оценка на (аритметичен) израз

Оценка на изрази

По време на лекции разработваме лексически анализатор, рекурсивен парсер и интерпретатор на прост език, състоящ се от аритметични изрази от следния вид:

```
<expression> ::= <number> | (<expression> <operator> <expression>)
<number> ::= {0,..,9}+
<operator> ::= + | - | * | /
```

За двата вида възможни изрази – числова константа и приложение на двуместен аритметичен оператор, оградено в скоби – дефинираме съответ-

но възли в синтактично дърво. Дърво на израза построяваме от символен низ и оценяваме рекурсивно.

- 32.1. Интерпретаторът на изрази от лекции да се разшири с възможност за групиране на изрази в “блокове”, започващи с ключовата дума **begin** и завършващи с ключовата дума **end**:

$$\langle \text{expression} \rangle ::= \dots \mid \text{begin } (\langle \text{expression} \rangle;)+ \text{end}$$

Оценката на блок от изрази да се състои в последователно оценяване на всички изрази в блока и използване на онцеката на последния израз от блока като оценка на целия блок. Например: **begin 1; 2; 3; end** има оценка 3.

- 32.2. Езикът, разработен на лекции, да се разшири с условен оператор от вида:

$$\langle \text{expression} \rangle ::= \dots \mid \text{if } \langle \text{expression} \rangle \text{ then } \langle \text{expression} \rangle \text{ else } \langle \text{expression} \rangle$$

Оценката на изразите от този вид е оценката на **expthen** или **exprelse** в зависимост от оценката на **condition**.

- 32.3. Интерпретаторът, разработен на лекции, да се разшири с възможност за присвояване на стойност на променлива:

$$\begin{aligned} \langle \text{expression} \rangle &::= \dots \mid \langle \text{assignment} \rangle \\ \langle \text{assignment} \rangle &::= \text{set } \langle \text{variable} \rangle = \langle \text{expression} \rangle \end{aligned}$$

Примерни изрази на този език и съответния резултат от оценката им са:

```
begin
  set x = (3*(1+2));
  x;
end
```

Оценка: 9

```
begin
  set x = 10;
  set y = 5*5;
  ((x*y)*2);
end
```

Оценка: 500

- 32.4. (*) Интерпретаторът на аритметични изрази от лекции да се разшири с възможност за извеждане на стойност на израз на стандартния изход:

```
<expression> ::= ... | print <expression>
```

Оценката на израз от типа **print** е се намира като оценката на израза **е**, като освен това се произвежда извеждане на стандартния изход като страничен ефект. Всяка отделна стойност да бъде изведена на нов ред.

Пример: Оценката на израза **print 1** ще е 1 и ще доведе до извеждане на "1" на стандартния изход. Оценката на

```
begin
  print 1;
  print 2;
  print begin 3; 4; end;
end
```

ще доведе до извеждане на следните редове на стандартния изход:

```
1
2
4
```

33 Двоични дървета

33.1 Прости обхождания

- 33.1. Да се дефинира функция **count**, която намира броя на елементите на дърво.
- 33.2. Да се дефинира функция **countEvens**, който намира броя на елементите на дърво от числа, които са четни.
- 33.3. Да се дефинира функция **searchCount pred t**, който намира броя на елементите на дървото **t**, които удовлетворяват предиката **pred**.
Да се приложи **searchCount** за решаване на горните две задачи.
- 33.4. Да се дефинира функция **countLeaves**, който намира броя на листата в дърво.
- 33.5. Да се дефинира функция **maxLeaf**, която намира най-голямото по стойност листо на непразно дърво.
- 33.6. Да се дефинира функция **isOrdered**, проверява дали дадено дърво е двоично наредено. *Упътване: използвайте помощна функция*

33.2 Извеждане и сериализация

33.7. Да се дефинира функция `prettyPrint t`, отпечатаваща двоично дърво на стандартния изход по следния начин: (1) всеки наследник е вдясно от родителя си, (2) елементите на еднакво ниво в дървото се отпечатват на еднаква колона от екрана, (3) десните наследници са на предишен ред от родителя си и (4) левите наследници са следващ ред спрямо родителя си.

Например, дървото от Фигура 3.3 би изглеждало по следния начин (да се отпечатат и номерата на редовете на стандартния изход):

```
1:      25
2:    20
3:    15
4:  10
5:    5
```

Представяне на двоично дърво в “Scheme формат” наричаме еднозначно текстово представяне на структурата от данни, образувано по следните правила:

- Празното дърво се представя с низа “()”
- Нека е дадено дървото t с корен x , ляво поддърво t_l и дясно поддърво t_r . Ако s_l е представянето в “Scheme формат” на t_l , а s_r е представянето в “Scheme формат” на t_r , то низът “(x s_l s_r)” е представянето на дървото t , където “ x ”, “ s_l ” и “ s_r ” са съответните низове.

Например, дървото от Фигура 3.3 се представя по следния начин:

```
(10 (20 (25 () ())) (15 () ())) (5 () ()))
```

33.8. Да се дефинират функции за сериализация и де-сериализация на двоично дърво, съдържащо числа, като се използва “Scheme формат”.

Може ли да се сериализира дърво, съдържащо други дървета във възлите си (“дърво от дървета”)?

33.3 Обхождания

33.9. Да се реализира функция `leaves` намираща списък със стойностите на листата на дадено дърво.

- 33.10. Да се дефинира функция `findTrace t x`. Ако `x` е елемент на дървото, функцията да връща следата на `x` (според дефиницията на “следа”, обсъждана на лекции). Ако `x` не е елемент на дървото, функцията да връща низа “_”.

Пример: За дървото от Фигура 3.3, следата на елемента със стойност 25 е “RR”.

- 33.11. Да се дефинира функция `getat t i`, която намира i -тият пореден елемент на дървото при обхождане корен-ляво-дясно.

Пример: За дървото от Фигура 3.3, елементът с пореден номер 0 е 10, с номер 1 е 5, с номер 2 е 20 и т.н.

- 33.12. Нека е дадено двоично дърво, чиито възли съдържат символи (`Char`). Да се намери:

- а) Най-дългия символен низ, който може да се получи при последователно конкатениране на символите от корена на дървото по пътя до някое негово листо. На дървото от фигура 3.4 това е низа “adef”.
- б) Най-дългия символен низ, който може да се получи при конкатениране на символите по възлите на някое ниво в дървото, отляво надясно. На дървото от фигура 3.4 това е низа “ceg”.

- 33.13. Нека са дадени две дървета t_1 и t_2 . Да се напише функция `equals t1 t2`, която проверява дали двете дървета са еднакви (използвайте интуитивна дефиниция за еднаквост на дървета).

- 33.14. Нека са дадени две дървета t_1 и t_2 . Да се напише функция `subtree t1 t2`, която проверява дали някое поддърво на t_1 е еднакво с t_2 . Вж. фигури 3.4 и 3.5.

33.4 Decision Tree

Decision Tree

На лекции разглеждаме двоични дървета, в чиито вътрешни възли се съдържа въпрос, а в листата – имена на класове от обекти. Такива дървета са вариант на т.нар. “класификационни дървета” (“decision trees”), които служат за класификация на обекти спрямо техни атрибути.

```
data Node = Class String |
           Attr {question :: String,
                  yes    :: Node,
```



```
no :: Node}
deriving Show
```

33.15. Нека е дадено класификационно дърво `t`. Да се дефинират функции:

- `questions :: Node -> [String]`, която връща списък с всички въпроси, които се задават в дървото.
- `classes :: Node -> ([Maybe Bool], String)`, която връща списък с всички класове в дървото, като за всеки клас са дадени отговорите на всички въпроси в реда, в който ги събира функцията `questions`. Ако за някой клас отговорът на даден въпрос не е известен, съответния елемент на списъка е `Nothing`.

Например, за дървото от Фигура 4.3, `questions t` връща

`["Има ли пера?", "Може ли да лети?", "Има ли перки?"]`,

а `classes t` връща:

```
(([Just True,Just True,Nothing],"Гълъб"),
 ([Just True,Just False,Nothing],"Пингвин"),
 ([Just False,Nothing,Just True],"Делфин"),
 ([Just False,Nothing,Just False],"Куче"])
```



Фигура 4.3: Дърво за класификация на животински видове

33.16. Нека е дадено класификационно дърво `t`. Да се дефинира функция `matches :: [Node] -> [(String, [String])]`, която за всеки клас в дървото връща двойка, съдържаща класа и списък с всички въпроси, за които отговорът е `True` за този клас. За дървото от Фигура 4.3, получаваме:

```
(("Гълъб",["Има ли пера?","Може ли да лети?"]),
 ("Пингвин",["Има ли пера?"]),
 ("Делфин",["Има ли перки?"]),
 ("Куче",[]])
```

- 33.17. Даден е списък с въпроси `questions::[String]` и списък с двойки от класове и отговорът на всеки въпрос за съответния клас, ако е известен. Ако за даден клас не е известен отговорът на някой въпрос, това се обозначава с `Nothing`. Например:

```
questions = ["Има ли пера?",
             "Може ли да лети?",
             "Има ли перки?"]
classes = [(Just True,Just True,Nothing),"Гълъб"),
           (Just True,Just False,Nothing),"Пингвин"),
           (Just False,Nothing,Just True),"Делфин"),
           (Just False,Nothing,Just False),"Куче")]
```

Да се дефинира функция `buildTree :: [String] -> [(Just Bool),String]` `-> Node`, която по зададен списък с въпроси и списък с класове, създава класификационно дърво за дадените класове при следните условия:

- Съществува число n , такова че броя въпроси е точно $2^n - 1$
- Броя класове е точно 2^n
- За всеки клас има точно n въпроса с известен отговор
- Няма два класа с еднакви отговори на въпросите

За списъците от примера, резултатното дърво е същото като на Фигура 4.3.

Упътване: За даден списък с класове, коренът на дървото за класификацията им ще съдържа този въпрос, отговорът на който е известен за всички класове. При избор на корен на дървото, как да разбием оставащите въпроси на две части – за лявото и за дясното поддърво?

- 33.18. Даден е списък с двойки от класове и списък с въпроси, които са верни за тези класове. Например:

```
(("Гълъб",["Има ли пера?","Може ли да лети?"]),
 ("Пингвин",["Има ли пера?"]),
 ("Делфин",["Има ли перки?"]),
 ("Куче",[]))
```

Ако са налице следните условия:

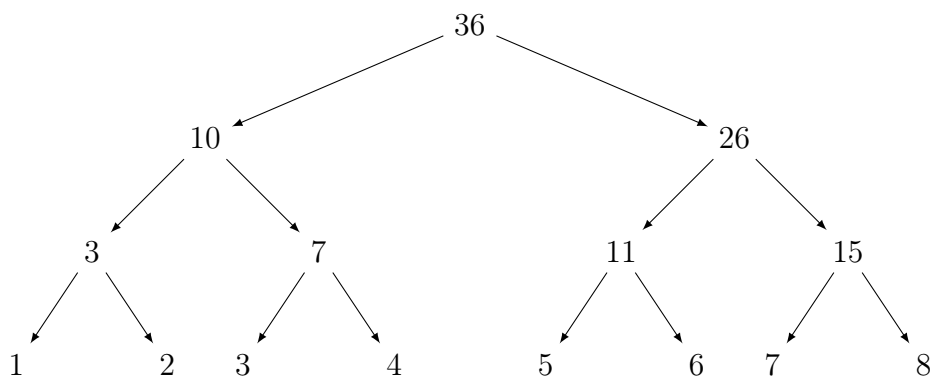
- Съществува число n , такова че броят въпроси е точно $2^n - 1$
- Броя класове е точно 2^n

- За всяко число i от 0 до $n - 1$ има точно $i + 1$ въпроса, които са верни за точно 2^{n-i} класа.

Да се построи класификационно дърво за дадените класове и въпроси. За списъците от примера, резултатното дърво е същото като на Фигура 4.3.

Как избиравме въпроса за корена на дървото? Кои класове остават в лявото и кои - в дясното поддърво?

33.5 Други



Фигура 4.4: Дърво, на което стойността на всеки връх е сума от стойностите на децата му

- 33.19. За дадено естествено число n , да се построи дърво, чиито листа, разгледани от ляво надясно, имат стойности $1, \dots, 2^n$, а всеки възел има стойност сумата на стойностите на децата си. Например, за $n = 3$, дървото е показано на Фигура 4.4.

Упътване: Инициализирайте опашка с листа $1, \dots, 2^n$. За всяка двойка излечени елементи от опашката, поставете обратно в нея поддърво с корен, обединяващ двата елемента. Повтаряйте операцията докато опашката има поне два елемента.

34 Наследяване

- 34.1. В духа на шаблона за дизайн “Итератор” да се разработи итератор за списъци, двойки и тройки.

- Да се дефинира типов клас `Iterator`, дефиниращ итератор над произволен съставен тип с операции `get` и `next` и предикат `end`

- Да се реализира итератор за списък, двойка и тройка с елементи от еднакъв тип
- Да се дефинира функция `sumIter`, която събира елементите на някаква структура от данни по даден итератор
- Да се демонстрира с пример използването на `sumIter` за поне два различни вида структури
- Да се направи итератора на функтор и да се демонстрира с какво това би било полезно
- Да се направи итератор на апликативен функтор и да се демонстрира с какво това би било полезно
- Типовият клас `Iterator` да дефинира двупосочен итератор и да се създадат инстанции за всички видове итератори, описани по-горе. Връщането назад да бъде операция с константна сложност за всеки от итераторите

34.2. (*)Как може да се релизира итератор за двоично дърво?

Библиография

- [1] Магдалина Тодорова, Петър Армянов, Дафина Петкова, Калин Георгиев, “Сборник от задачи по програмиране на C++. Първа част. Увод в програмирането”
- [2] А. Дичев, И. Сосков, “Теория на програмите”, Издателство на СУ, София, 1998
- [3] Wikiphow, How to Draw Pusheen the Cat, <https://www.wikihow.com/Draw-Pusheen-the-Cat>
- [4] R. W. Floyd. “Assigning meanings to programs.” Proceedings of the American Mathematical Society, Symposia on Applied Mathematics. Vol. 19, pp. 19–31. 1967.
- [5] Александра Соскова, Стела Николова, “Теория на програмите в задачи”, Софтех, 2003
- [6] David Matuszek, “Backtracking”, <https://www.cis.upenn.edu/~matuszek/cit594-2012/Pages/backtracking.html>.
- [7] Магдалина Тодорова, Петър Армянов, Дафина Петкова, Калин Николов, “Сборник от задачи по програмиране на C++. Втора част. Обектно ориентирано програмиране”
- [8] Trapezoidal rule, https://en.wikipedia.org/wiki/Trapezoidal_rule